Homework 6: Non-negative FIR Filter Design

Name: Jon Knapp

Date: November 10, 2026

Class: EE 522 Discrete Time Processing

AI Disclosure:

AI was used in the following ways:

- Code generation
- Markdown generation
- Checking my work and my logic
- Process suggestions
- Project analysis to fit non-negative FIR filters

Problem 1: Select three different non-causal, non-negative impulse responses with even symmetry. The purpose of these filters is to smooth the signals they are applied to, where the amount of smoothing is controlled by the duration of the impulse response. Please scale the impulse responses for a DC gain of 1. Give a mathematical or verbal description of each impulse response, and provide your rationale for using this impulse response.

How I'm interpreting the requirements:

Each filter is non-causal and centered at $n = 0$, defined on $-M \leq n \leq M$ with length $N = 2M+1$. All three satisfy:

- **Non-negative:** $h[n] \geq 0$ for all n
- **Even symmetry:** $h[n] = h[-n]$
- **Unit DC gain:** $H(e^{j0}) = \sum_{n=-M}^{M} h[n] = 1$

The half-length M sets the duration of the impulse response, and therefore the amount of smoothing.

Filter 1: Rectangular (moving average / boxcar)

$$h_{\text{rect}}[n] = \begin{cases} \dfrac{1}{2M+1}, & |n| \leq M \\ 0, & \text{otherwise} \end{cases}$$

Every sample in the window receives equal weight, so the output is the arithmetic mean of the $2M+1$ input samples centered on n.

DC gain: $\displaystyle\sum_{n=-M}^{M} \frac{1}{2M+1} = \sum_{n=-M}^{-1} \frac{1}{2M+1} + \frac{1}{2M+1} + \sum_{n=1}^{M} \frac{1}{2M+1} = \frac{M}{2M+1} + \frac{1}{2M+1} + \frac{M}{2M+1} = \frac{2M+1}{2M+1} = 1$

Frequency response. Start from the DTFT definition. Only the $2M+1$ nonzero taps contribute:

$$H_{\text{rect}}(e^{j\omega}) = \sum_{n=-\infty}^{\infty} h_{\text{rect}}[n]\, e^{-j\omega n} = \frac{1}{2M+1} \sum_{n=-M}^{M} e^{-j\omega n}$$

Step 1: Shift the index so the sum starts at 0. Let $k = n + M$, so $n = k - M$ and k runs from 0 to $2M$:

$$H_{\text{rect}}(e^{j\omega}) = \frac{1}{2M+1} \sum_{k=0}^{2M} e^{-j\omega(k-M)} = \frac{e^{j\omega M}}{2M+1} \sum_{k=0}^{2M} \left(e^{-j\omega}\right)^k$$

Step 2: Sum the finite geometric series. With ratio $r = e^{-j\omega}$ and $2M+1$ terms, $\sum_{k=0}^{2M} r^k = \dfrac{1-r^{2M+1}}{1-r}$ for $r \neq 1$ (that is, $\omega \neq 0$):

$$H_{\text{rect}}(e^{j\omega}) = \frac{e^{j\omega M}}{2M+1} \cdot \frac{1 - e^{-j\omega(2M+1)}}{1 - e^{-j\omega}}$$

Step 3: Factor out half-angle exponentials. Pull out the exponential at half of each exponent, so that what remains is a difference of conjugates. Then use $e^{j\theta} - e^{-j\theta} = 2j\sin\theta$:

$$1 - e^{-j\omega(2M+1)} = e^{-j\omega(2M+1)/2}\left(e^{j\omega(2M+1)/2} - e^{-j\omega(2M+1)/2}\right) = e^{-j\omega(2M+1)/2}\, 2j\sin(\omega(2M+1)/2)$$

$$1 - e^{-j\omega} = e^{-j\omega/2}\left(e^{j\omega/2} - e^{-j\omega/2}\right) = e^{-j\omega/2}\, 2j\sin(\omega/2)$$

Step 4: Substitute and simplify. The $2j$ factors cancel, and the remaining exponentials combine to 1:

$$H_{\text{rect}}(e^{j\omega}) = \frac{1}{2M+1} \cdot \underbrace{e^{j\omega M}\, e^{-j\omega(2M+1)/2}\, e^{j\omega/2}}_{e^{j\omega(M-M-1/2+1/2)} = 1} \cdot \frac{\sin(\omega(2M+1)/2)}{\sin(\omega/2)}$$

so

$$H_{\text{rect}}(e^{j\omega}) = \frac{1}{2M+1} \frac{\sin(\omega(2M+1)/2)}{\sin(\omega/2)}$$

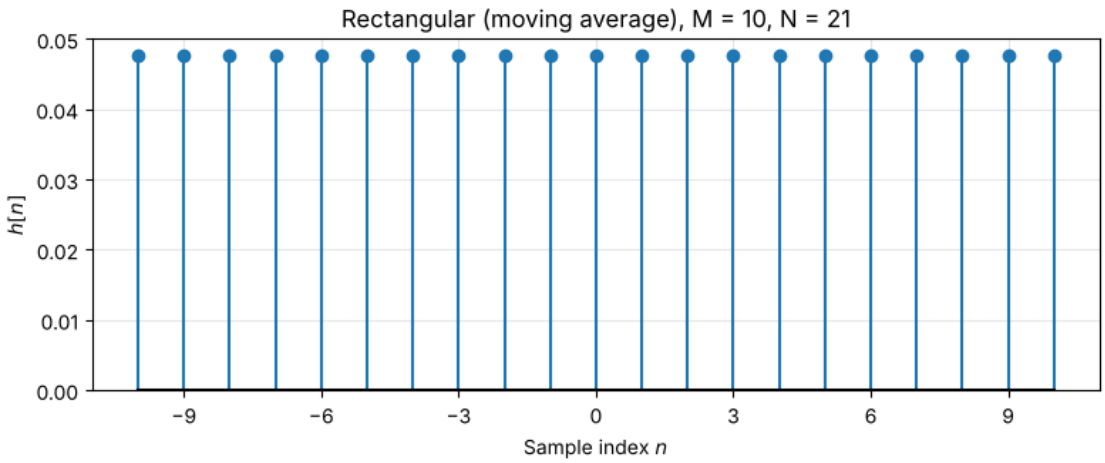
This is the Dirichlet kernel. The complex exponentials cancel because the filter is centered at $n = 0$; this is the even symmetry $h[n] = h[-n]$ at work. As a result, $H_{\text{rect}}(e^{j\omega})$ is purely real.

Check at DC: As $\omega \to 0$, $\sin(\omega(2M+1)/2) \approx \omega(2M+1)/2$ and $\sin(\omega/2) \approx \omega/2$, so $H_{\text{rect}}(e^{j\omega}) \to 1$. This matches the DC gain above.

Phase: $H_{\text{rect}}(e^{j\omega})$ is real but not always positive. It is positive in the main lobe, $|\omega| < 2\pi/(2M+1)$, and changes sign at each zero, $\omega = 2\pi k/(2M+1)$. The phase is therefore 0 where $H_{\text{rect}} > 0$ and π where $H_{\text{rect}} < 0$.

Rationale: The simplest possible smoother and the baseline for comparison.

```
Rectangular   N =  21   sum(h) = 1.000000000000   min(h) = 4.762e-02   symmetric
= True
```



Filter 2: Triangular (Bartlett)

$$h_{\text{tri}}[n] = \begin{cases} \dfrac{M+1-|n|}{(M+1)^2}, & |n| \le M \\ 0, & \text{otherwise} \end{cases}$$

The weights fall off linearly from a peak of $\frac{1}{M+1}$ at $n = 0$ and reach zero at $|n| = M + 1$, so all $2M + 1$ taps in the window are strictly positive.

DC gain:

$$\sum_{n=-M}^{M} (M+1-|n|) = (M+1) + 2\sum_{k=1}^{M}(M+1-k) = (M+1) + M(M+1) = (M+1)^2$$

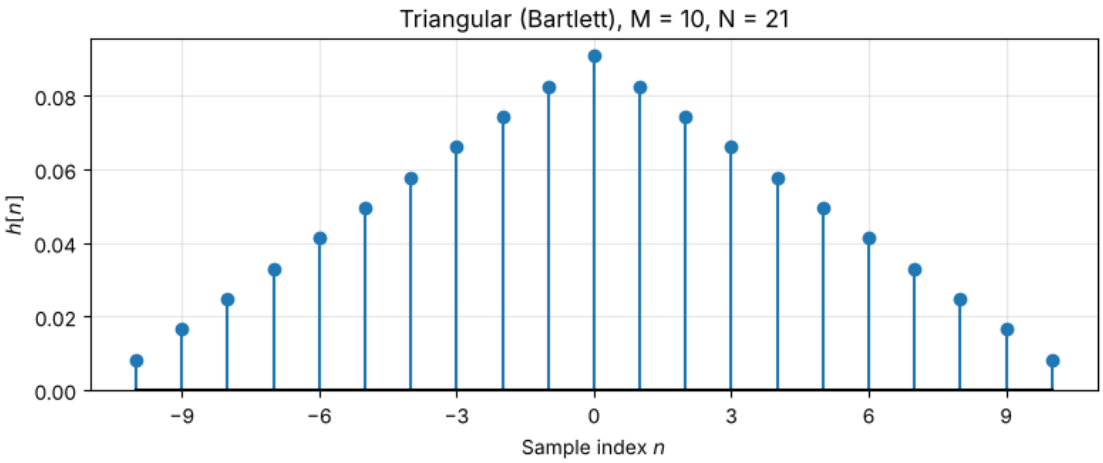
so dividing by $(M + 1)^2$ gives a sum of 1.

Frequency response: The triangle equals the convolution of two length-$(M + 1)$ rectangles, $h_{\text{tri}} = h_{\text{rect},M+1} * h_{\text{rect},M+1}$, so its response is the square of a Dirichlet kernel:

$$H_{\text{tri}}(e^{j\omega}) = \left[\frac{1}{M+1}\,\frac{\sin(\omega(M+1)/2)}{\sin(\omega/2)}\right]^2 \ge 0$$

Rationale: Tapering the edges trades a wider main lobe for much lower sidelobes (about −26 dB), and because $H_{\text{tri}}(e^{j\omega})$ is never negative, the filter never inverts the sign of any frequency component.

```
Triangular    N =  21    sum(h) = 1.000000000000    min(h) = 8.264e-03    symmetric
= True
```



Filter 3: Binomial

$$h_{\text{bin}}[n] = \begin{cases} \dfrac{1}{2^{2M}} \dbinom{2M}{M+n}, & |n| \le M \\ 0, & \text{otherwise} \end{cases}$$

The taps are row $2M$ of Pascal's triangle. Equivalently, h_{bin} is the two-tap average $[\frac{1}{2}, \frac{1}{2}]$ convolved with itself $2M$ times (then centered at $n = 0$).

THE FOLLOWING DERIVATION WAS DONE WITH AI

DC gain. The DC gain is the frequency response at $\omega = 0$, which is the sum of the taps:

$$H_{\text{bin}}(e^{j0}) = \sum_{n=-M}^{M} h_{\text{bin}}[n] = \frac{1}{2^{2M}} \sum_{n=-M}^{M} \binom{2M}{M+n}$$

Step 1: Shift the index so the sum starts at 0. Let $k = M + n$, so k runs from 0 to $2M$:

$$H_{\text{bin}}(e^{j0}) = \frac{1}{2^{2M}} \sum_{k=0}^{2M} \binom{2M}{k}$$

Step 2: Apply the binomial theorem. For any a and b, $(a+b)^N = \sum_{k=0}^{N} \binom{N}{k} a^k b^{N-k}$. With $a = b = 1$ and $N = 2M$:

$$\sum_{k=0}^{2M} \binom{2M}{k} = (1+1)^{2M} = 2^{2M}$$

Step 3: Divide by the scale factor.

$$H_{\text{bin}}(e^{j0}) = \frac{2^{2M}}{2^{2M}} = 1$$

The factor $1/2^{2M}$ is chosen exactly to cancel the sum of row $2M$ of Pascal's triangle.

Frequency response. Start from the DTFT definition. Only the $2M+1$ nonzero taps contribute:

$$H_{\text{bin}}(e^{j\omega}) = \sum_{n=-\infty}^{\infty} h_{\text{bin}}[n]\, e^{-j\omega n} = \frac{1}{2^{2M}} \sum_{n=-M}^{M} \binom{2M}{M+n} e^{-j\omega n}$$

Step 1: Shift the index so the sum starts at 0. Let $k = n + M$, so $n = k - M$ and k runs from 0 to $2M$:

$$H_{\text{bin}}(e^{j\omega}) = \frac{1}{2^{2M}} \sum_{k=0}^{2M} \binom{2M}{k} e^{-j\omega(k-M)} = \frac{e^{j\omega M}}{2^{2M}} \sum_{k=0}^{2M} \binom{2M}{k} \left(e^{-j\omega}\right)^k$$

Step 2: Apply the binomial theorem. With $a = e^{-j\omega}$, $b = 1$, and $N = 2M$, the sum collapses to a single power:

$$\sum_{k=0}^{2M} \binom{2M}{k} \left(e^{-j\omega}\right)^k = \left(1 + e^{-j\omega}\right)^{2M} \quad \Longrightarrow \quad H_{\text{bin}}(e^{j\omega}) = e^{j\omega M} \left(\frac{1 + e^{-j\omega}}{2}\right)^{2M}$$

The term in parentheses is the DTFT of the two-tap average $\left[\frac{1}{2}, \frac{1}{2}\right]$. Raising it to the power $2M$ is the convolution theorem at work: convolving that average with itself $2M$ times multiplies its response by itself $2M$ times.

Step 3: Factor out the half-angle exponential. Pull out $e^{-j\omega/2}$, so that what remains is a sum of conjugates. Then use $e^{j\theta} + e^{-j\theta} = 2\cos\theta$:

$$1 + e^{-j\omega} = e^{-j\omega/2}\left(e^{j\omega/2} + e^{-j\omega/2}\right) = e^{-j\omega/2}\, 2\cos(\omega/2) \quad \Longrightarrow \quad \frac{1 + e^{-j\omega}}{2} = e^{-j\omega/2}\cos(\omega/2)$$

The factor $e^{-j\omega/2}$ is the half-sample delay of one two-tap stage.

Step 4: Substitute and simplify. Raising to the power $2M$ turns the half-sample delays into $e^{-j\omega M}$, which cancels the $e^{j\omega M}$ from Step 1:

$$H_{\text{bin}}(e^{j\omega}) = \underbrace{e^{j\omega M}\, e^{-j\omega M}}_{=1} \cos^{2M}\left(\frac{\omega}{2}\right)$$

so

$$H_{\text{bin}}(e^{j\omega}) = \cos^{2M}\left(\frac{\omega}{2}\right)$$

As with the rectangular filter, the complex exponentials cancel because the filter is centered at $n = 0$, so $H_{\text{bin}}(e^{j\omega})$ is purely real.

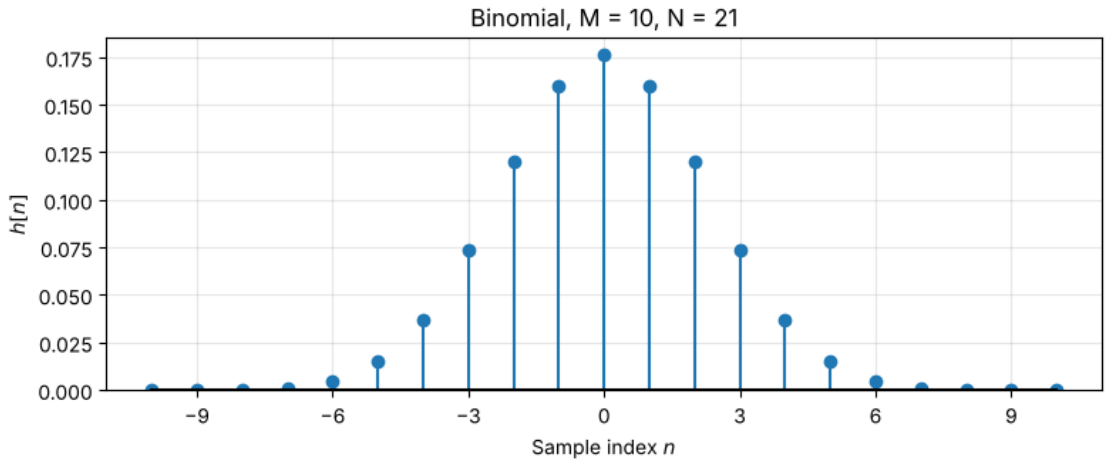
Check at DC: $\cos^{2M}(0) = 1$, which matches the DC gain above.

Properties:

- **Non-negative:** the exponent $2M$ is even, so $H_{\text{bin}}(e^{j\omega}) \geq 0$ for every ω. The phase is therefore 0 at all frequencies (it is undefined only at $\omega = \pi$, where $H_{\text{bin}} = 0$).
- **Monotonic:** on $0 \leq \omega \leq \pi$, $\cos(\omega/2)$ falls steadily from 1 to 0 and is never negative, so $\cos^{2M}(\omega/2)$ also falls steadily from 1 at $\omega = 0$ to 0 at $\omega = \pi$. There are no sidelobes and no ripple.

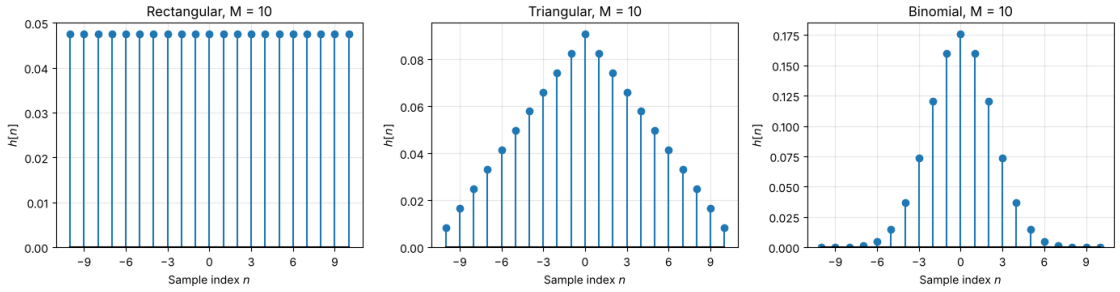
Rationale: Binomial filters are used a lot in image processing because it's the cost-effective solution for wanting to use a Gaussian filter. It's being tested here to allow me to further investigate it.

```
Binomial      N =  21   sum(h) = 1.000000000000   min(h) = 9.537e-07   symmetric
= True
```



Side-by-side comparison

All three filters with the same half-length M. Note the different vertical scales: the taps in each filter always sum to 1, so a more concentrated filter has a taller peak.



Problem 2: For each of the three filters, provide the following:

- An explanation of how you enforce the requirement that the DC gain is 1
- A plot that shows how the cutoff frequency of the filter is related to the duration of the filter
- For each of the cutoff frequencies of 0.05 pi, 0.1 pi, and 0.2 pi, include the following in your report:
  - Magnitude response
  - Phase response
  - Group delay
  - Impulse response
  - Time domain example of the input and output signals applied to the following signals. Please include the input and output signals on the same plot with clear labels that make

it clear which signal is the input and which is the output.
    * A white noise, signal with 200 samples, a mean of 1, and a variance of 1. Be mindful of edge conditions.
    * A shifted step x[n] = u[n-100] with 200 samples. Be mindful of edge conditions.
    * 100 samples of a sinusoid with a frequency of 0.1 pi radians per sample.
- Power spectral density plot of the input signal
    * ***Please use the periodogram method or an appropriately scaled FFT to compute the PSD on all signals.***
- Power spectral density plot of the output signal
    * ***Please use the periodogram method or an appropriately scaled FFT to compute the PSD on all signals.***

An explanation of how you enforce the requirement that the DC gain is 1

Filter 1: Rectangular (moving average / boxcar)

$$h_{\text{rect}}[n] = \begin{cases} \dfrac{1}{2M+1}, & |n| \le M \\ 0, & \text{otherwise} \end{cases}$$

DC gain: $\displaystyle\sum_{n=-M}^{M} \frac{1}{2M+1} = \frac{2M+1}{2M+1} = 1$

Filter 2: Triangular (Bartlett)

$$h_{\text{tri}}[n] = \begin{cases} \dfrac{M+1-|n|}{(M+1)^2}, & |n| \le M \\ 0, & \text{otherwise} \end{cases}$$

DC gain:

$$\sum_{n=-M}^{M} (M+1-|n|) = (M+1) + 2\sum_{k=1}^{M}(M+1-k) = (M+1) + M(M+1) = (M+1)^2$$

so dividing by $(M+1)^2$ gives a sum of 1.

Filter 3: Binomial

$$h_{\text{bin}}[n] = \begin{cases} \dfrac{1}{2^{2M}}\dbinom{2M}{M+n}, & |n| \le M \\ 0, & \text{otherwise} \end{cases}$$

DC gain: By the binomial theorem,

$$\sum_{k=0}^{2M} \binom{2M}{k} = (1+1)^{2M} = 2^{2M}$$

so dividing by 2^{2M} gives a sum of exactly 1.

A plot that shows how the cutoff frequency of the filter is related to the duration of the filter

Cutoff definition. I use the -3 dB (half-power) frequency: the smallest $\omega_c > 0$ where

$$|H(e^{j\omega_c})| = \frac{1}{\sqrt{2}}.$$

Because each $h[n]$ is even-symmetric about $n = 0$, the frequency response is purely real:

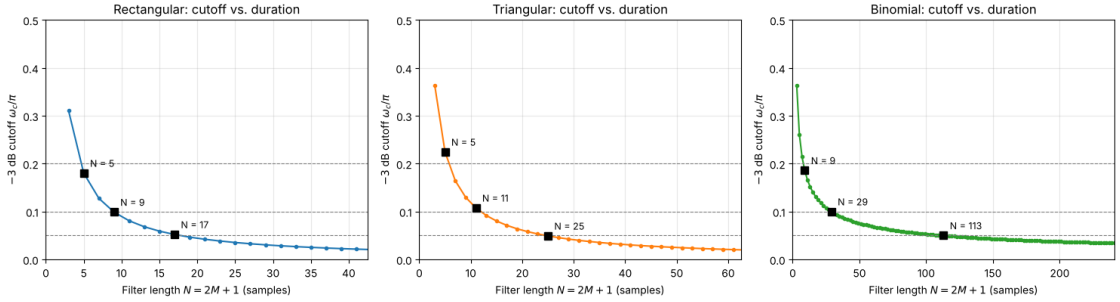
$$H(e^{j\omega}) = h[0] + 2\sum_{n=1}^{M} h[n]\cos(\omega n),$$

so I evaluate it on a dense grid over $[0, \pi]$, find the first grid point that drops below $1/\sqrt{2}$, and refine the crossing with a root finder.

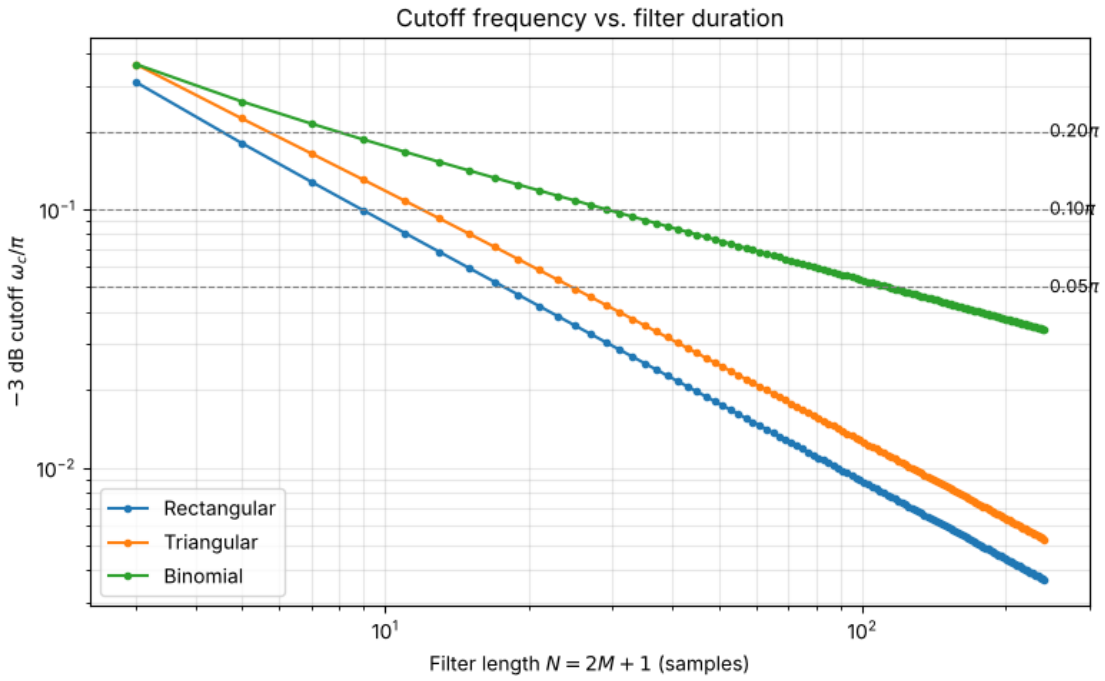
Duration is the number of taps, $N = 2M + 1$. Since M must be an integer, only a discrete set of cutoffs is achievable, so for each target cutoff I pick the M whose cutoff is closest.

Cutoff vs. duration for each filter

Each plot shows the -3 dB cutoff (in units of π) against the filter length N. The dashed lines mark the three target cutoffs, and the marked points are the lengths that come closest to each target.



All three filters on log-log axes



Filter lengths chosen for each target cutoff

Filter	Target	M	N	Actual wc/pi	Error
Rectangular	0.05pi	8	17	0.0522pi	+4.4%
Rectangular	0.10pi	4	9	0.0990pi	-1.0%
Rectangular	0.20pi	2	5	0.1803pi	-9.8%
Triangular	0.05pi	12	25	0.0492pi	-1.6%
Triangular	0.10pi	5	11	0.1077pi	+7.7%
Triangular	0.20pi	2	5	0.2245pi	+12.3%
Binomial	0.05pi	56	113	0.0501pi	+0.1%
Binomial	0.10pi	14	29	0.1000pi	-0.0%
Binomial	0.20pi	4	9	0.1860pi	-7.0%

For each filter and target cutoff, I use the length $N = 2M + 1$ chosen above and show three figures:

1. **Filter characteristics:** impulse response, magnitude, phase, and group delay.
2. **Time-domain examples:** input and output for white noise, the shifted step, and the sinusoid.
3. **Power spectral densities (PSDs):** input and output for each test signal.

Phase and group delay

Each filter is even-symmetric about $n = 0$, so $H(e^{j\omega}) = A(\omega)$ is real. A real number has a phase of 0 when it is positive and $\pm\pi$ when it is negative:

$$\angle H(e^{j\omega}) = \begin{cases} 0, & A(\omega) > 0 \\ \pm\pi, & A(\omega) < 0 \end{cases}$$

- **Triangular and binomial:** $A(\omega) \geq 0$, so the phase is always 0.
- **Rectangular:** the negative sidelobes cause jumps of π.
- **Group delay:** zero at every frequency, so the output is not shifted in time.
- **Causal version:** implementing the filter as $h[n - M]$ adds a linear phase of $-M\omega$ and a constant delay of M samples (the dashed curves).

Phase and group delay are left blank where $|H| < -200$ dB, because there the response is just roundoff noise. This only affects the long binomial filters.

Edge handling

What is an edge region? To compute the output $y[n]$, the filter needs the inputs $x[n - M]$ through $x[n + M]$: M samples on each side of n. The signal only has samples $0 \leq n \leq L - 1$, so near the ends some of the needed samples do not exist. An output sample has a complete window only when

$$n - M \geq 0 \quad \text{and} \quad n + M \leq L - 1 \qquad \Longleftrightarrow \qquad M \leq n \leq L - 1 - M.$$

Every other output sample is in the **edge region**, which is shaded gray on the plots.

Example (binomial, $M = 56$, $N = 113$):

- **White noise and step ($L = 200$):** windows are complete only for $56 \leq n \leq 143$. The first 56 and last 56 samples are shaded.
- **Sinusoid ($L = 100$):** a complete window would need $56 \leq n \leq 43$, which is impossible. The filter (113 samples) is longer than the signal (100 samples), so *every* output sample has a truncated window, and the whole plot is shaded.

What to do about the missing samples. Zero padding assumes the missing samples are 0. That pulls the output toward 0 near the edges, which is badly wrong for the mean-1 noise and the step. Instead I use **normalized convolution**: I use only the samples that exist, then divide by the sum of the taps that were used:

$$y[n] = \frac{\sum_k h[k]\, x[n-k]}{\sum_k h[k]}, \qquad \text{both sums over } k \text{ with } 0 \leq n - k \leq L - 1.$$

Example at $n = 0$: the window covers $-56 \leq n \leq 56$, but only $x[0], \ldots, x[56]$ exist. The taps used are the center tap and one side, which sum to about 0.54. Dividing by 0.54 rescales those taps to sum to 1. The output is then a true weighted average of $x[0], \ldots, x[56]$, rather than an average in which 46% of the weight lands on zeros.

Because $h[n] \geq 0$, the denominator is always positive. A constant input therefore gives exactly the same constant output everywhere, including at the edges, so the DC gain stays 1. In the interior, the denominator is 1 and this is ordinary convolution.

THE FOLLOWING DERIVATION WAS DONE WITH AI

PSD estimation

Every PSD is a **periodogram**: one FFT over the entire record, with no segmenting and no averaging. Welch's method fails for the step, because most of its segments would be all 0s or all 1s.

$$\widehat{P}(\omega_k) = \frac{c_k\,|X[k]|^2}{2\pi L}, \qquad c_k = 2 \text{ except } c_k = 1 \text{ at } \omega = 0 \text{ and } \pi$$

- Dividing by $2\pi L$ makes the area under the PSD equal the signal's average power (Parseval's relation). The check below confirms this.
- Zero padding to 1024 points only smooths the plotted curve; it does not add resolution.
- The mean is kept, so the DC component is visible.
- For white noise, the dashed lines show the theory: σ^2/π for the input and $|H|^2\sigma^2/\pi$ for the output.

What to look for

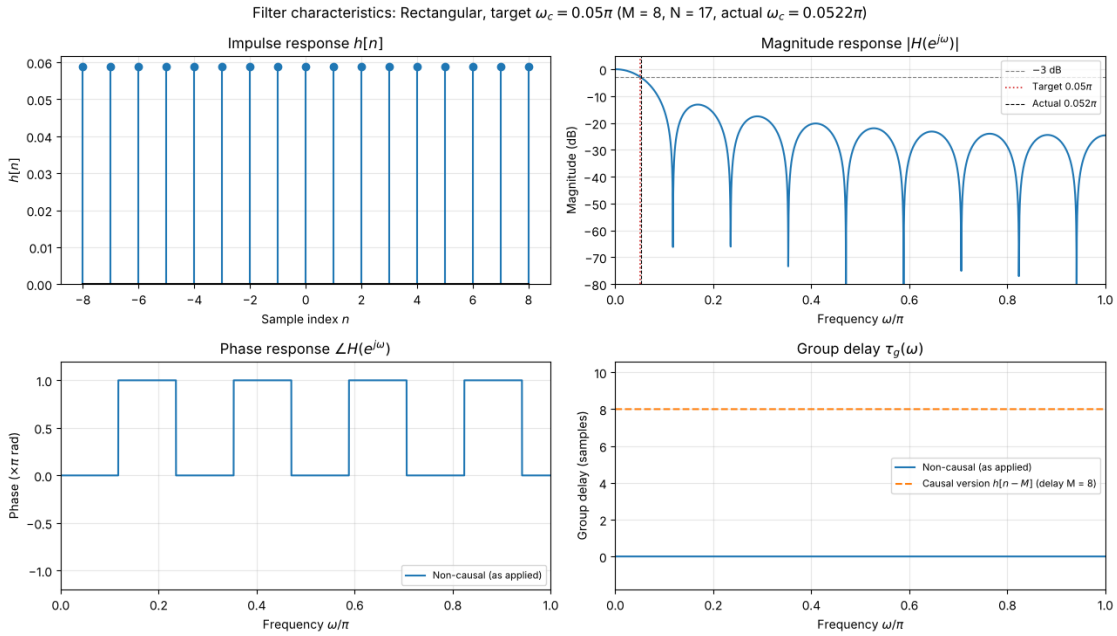
- **White noise:** a single periodogram is noisy (swings of about ± 10 dB are normal). The output follows the theory on average.
- **Leakage floor:** with no window, the periodogram effectively uses a rectangular window. The noise's DC component leaks across the band and creates a floor near -30 dB. That floor comes from the estimator, not the filter.
- **Step:** the periodogram sees the record as a $0 \to 1 \to 0$ pulse, like one period of a square wave, which gives closely spaced nulls in the input PSD. The filter smooths the jump at $n = 100$, leaving only the drop at the end of the record. The output PSD is therefore a smooth curve about 6 dB lower.
- **Sinusoid:** the input PSD peaks at 0.1π (5 full cycles in 100 samples). The output peak is scaled by $|H(e^{j0.1\pi})|^2$: it is heavily attenuated when the cutoff is below 0.1π and nearly unchanged when the cutoff is 0.2π.

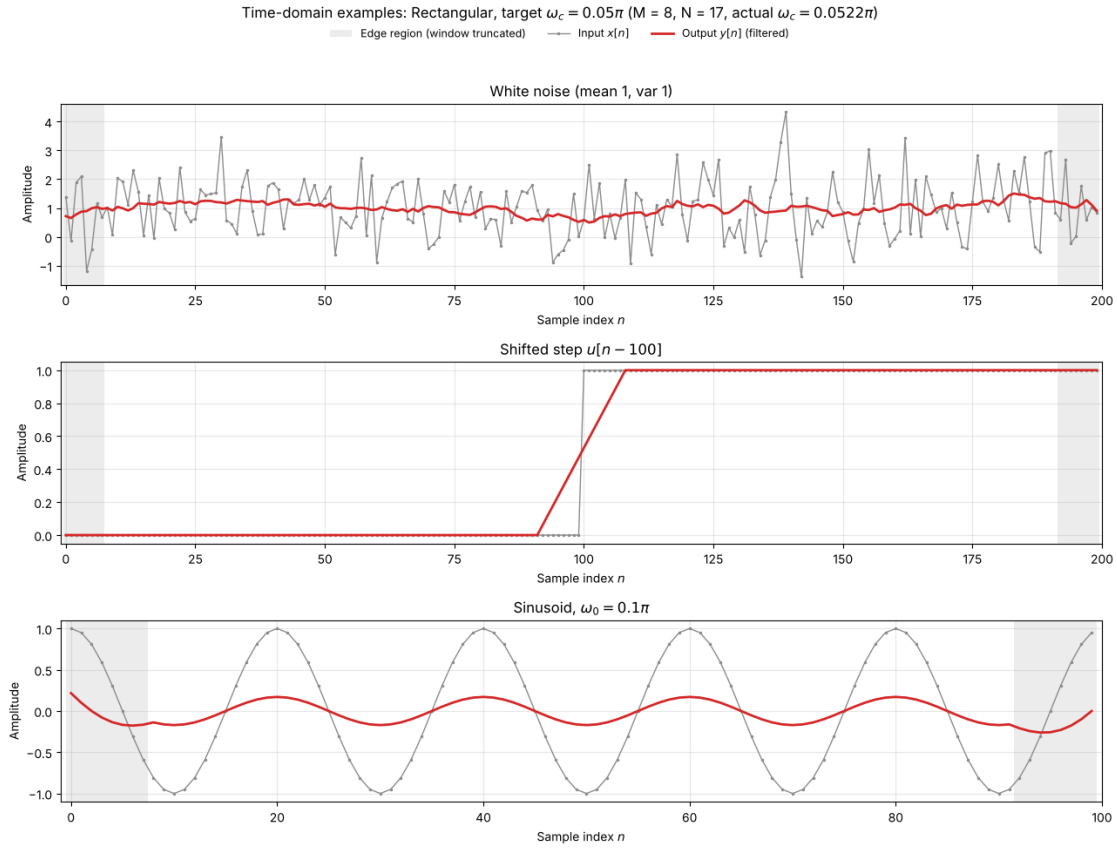
PSD scaling check This check confirms that `scipy.signal.periodogram` matches the hand-scaled FFT $c_k|X[k]|^2/(2\pi L)$, and that the area under each PSD equals the signal's average power.

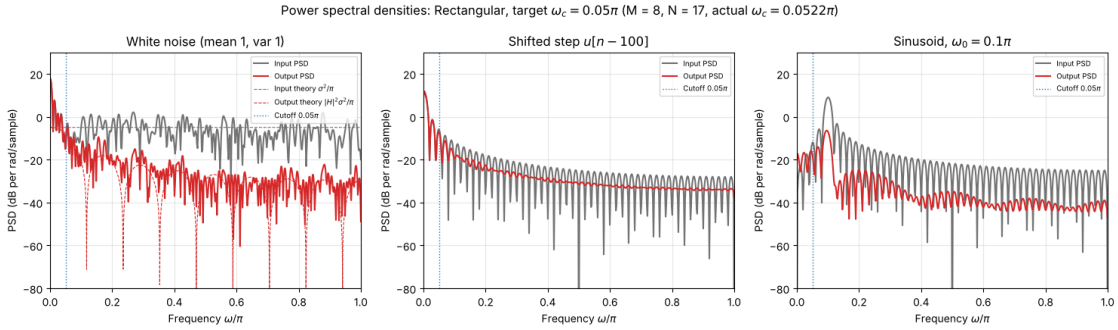
Signal	L	max |periodogram - FFT|	Area under PSD	mean(x^2)
White noise	200	2.13e-14	1.995000	1.995000
Shifted step	200	3.55e-15	0.500000	0.500000
Sinusoid	100	3.55e-15	0.500000	0.500000

Rectangular (moving average)

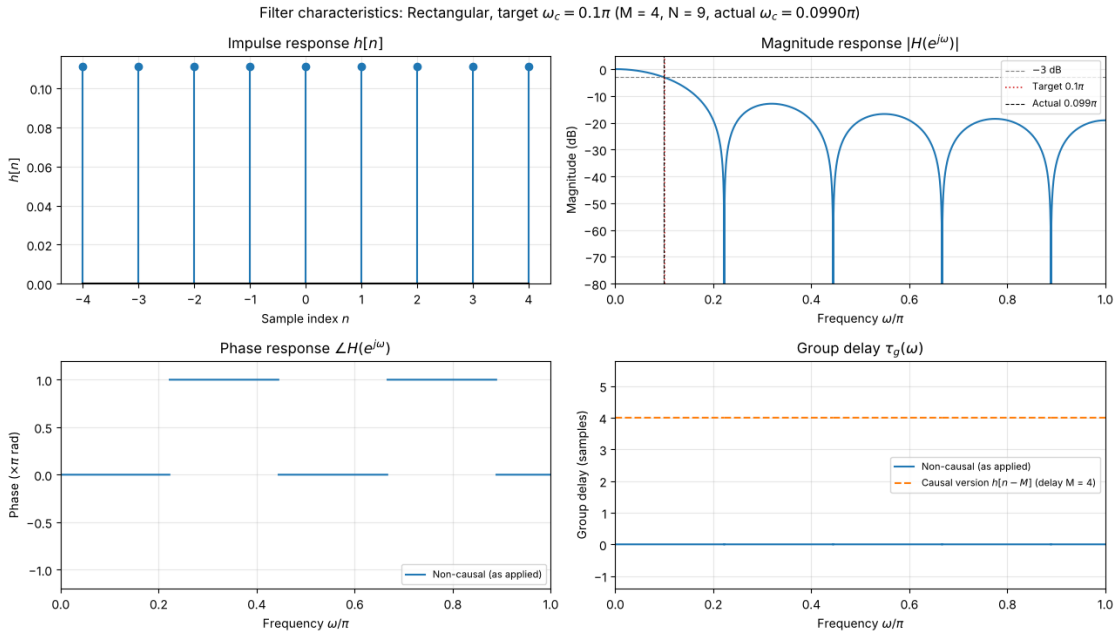
Rectangular, $\omega_c = 0.05\pi$

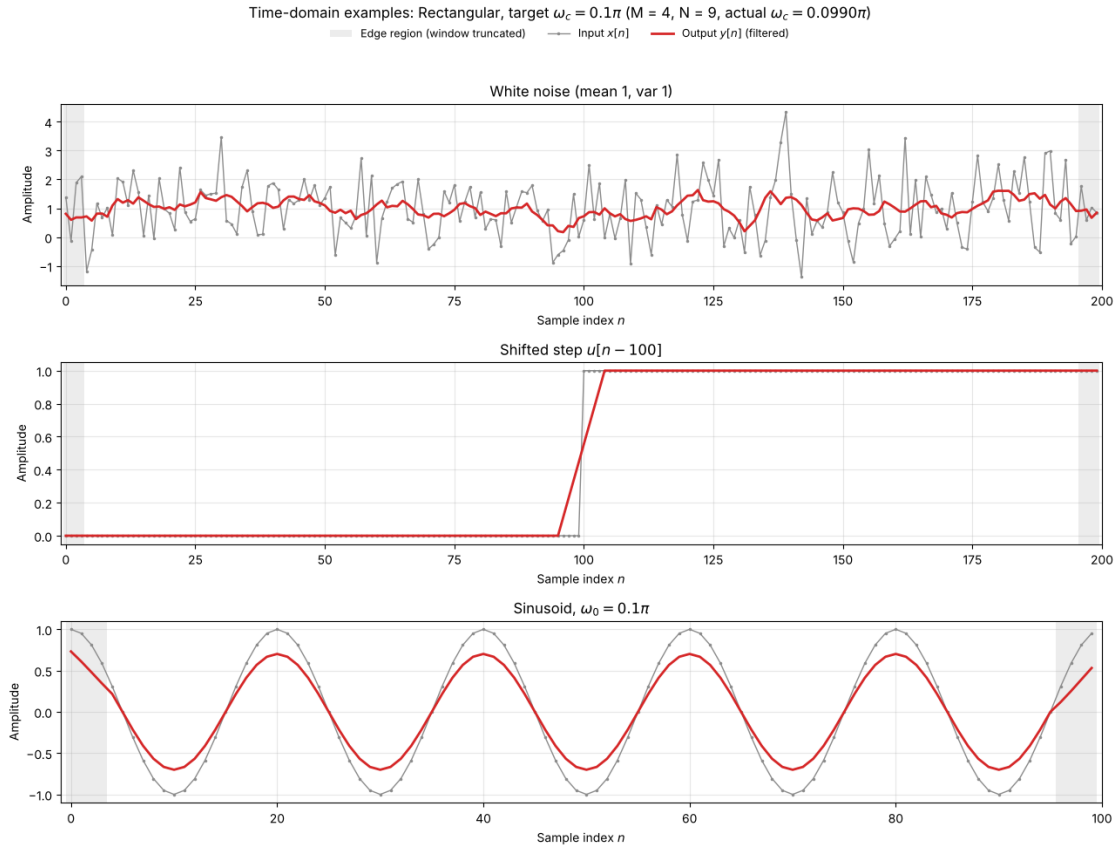

Filter characteristics: Rectangular, target $\omega_c = 0.05\pi$ (M = 8, N = 17, actual $\omega_c = 0.0522\pi$)
Impulse response $h[n]$
$h[n]$
Sample index n
Magnitude response $|H(e^{j\omega})|$
Magnitude (dB)
−3 dB
Target 0.05π
Actual 0.052π
Frequency ω/π
Phase response $\angle H(e^{j\omega})$
Phase (×π rad)
Non-causal (as applied)
Frequency ω/π
Group delay $\tau_g(\omega)$
Group delay (samples)
Non-causal (as applied)
Causal version $h[n-M]$ (delay M = 8)
Frequency ω/π

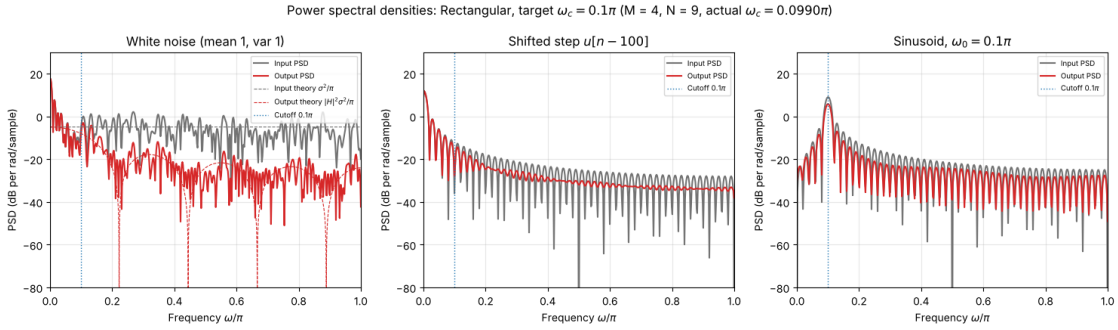

Time-domain examples: Rectangular, target $\omega_c = 0.05\pi$ (M = 8, N = 17, actual $\omega_c = 0.0522\pi$)
Edge region (window truncated)
Input $x[n]$
Output $y[n]$ (filtered)
White noise (mean 1, var 1)
Amplitude
Sample index n
Shifted step $u[n-100]$
Amplitude
Sample index n
Sinusoid, $\omega_0 = 0.1\pi$
Amplitude
Sample index n



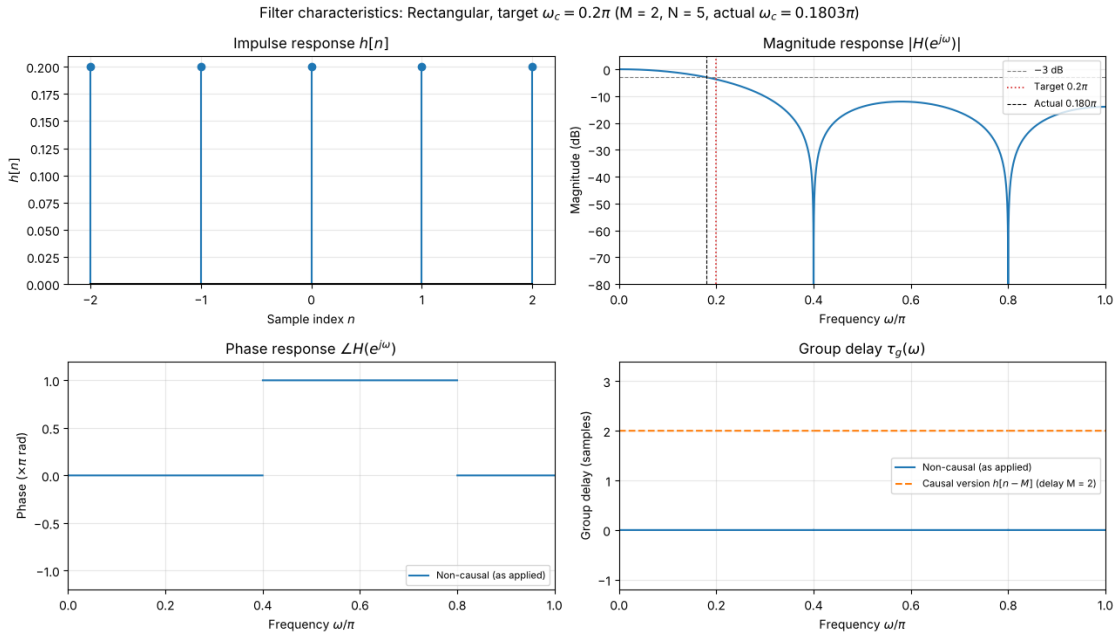
Rectangular, $\omega_c = 0.1\pi$

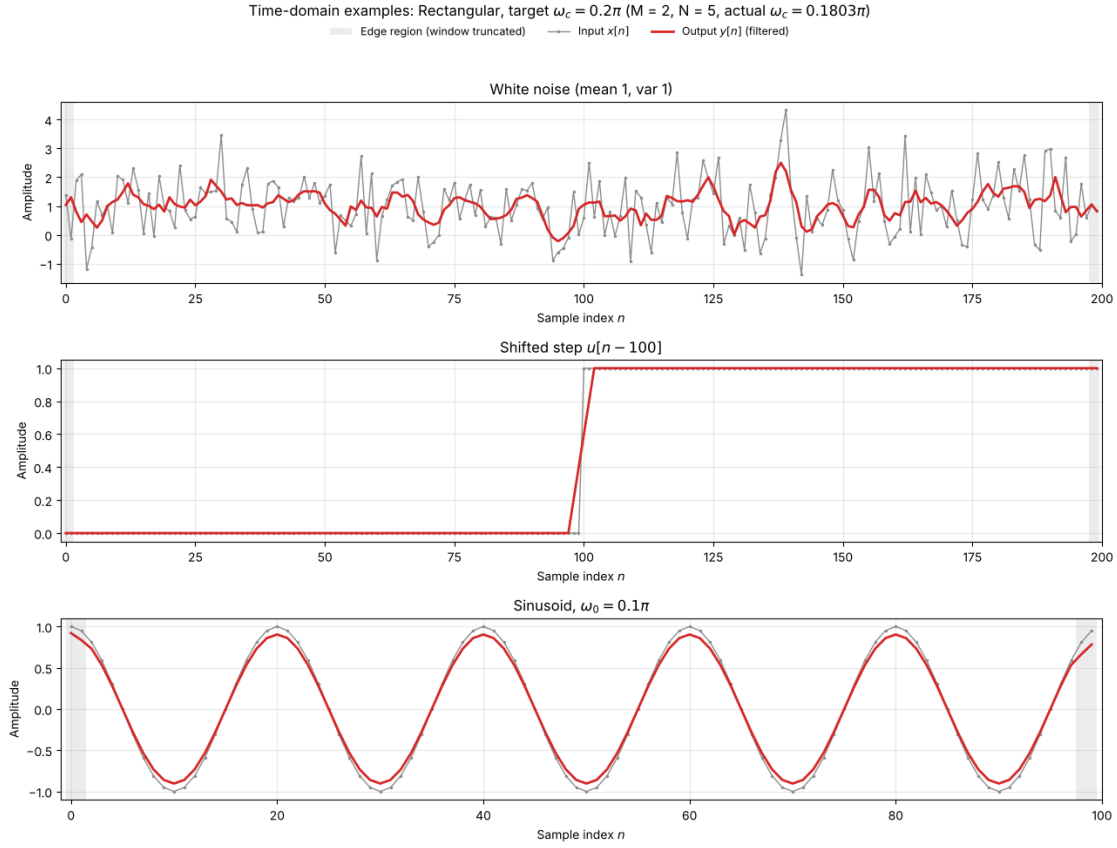


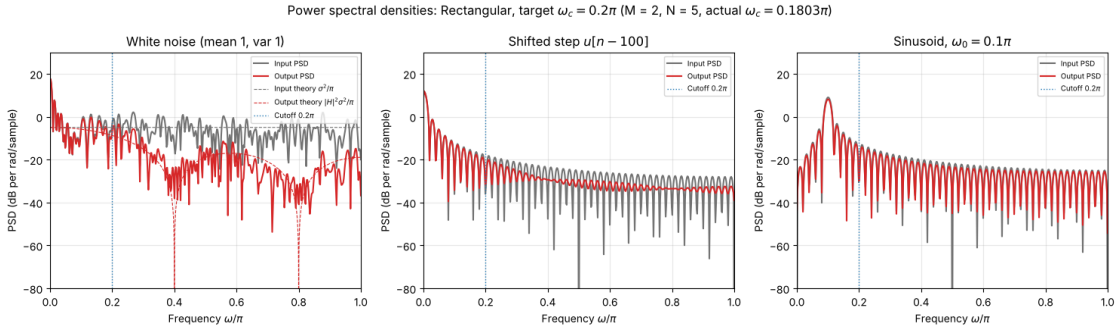




Rectangular, $\omega_c = 0.2\pi$

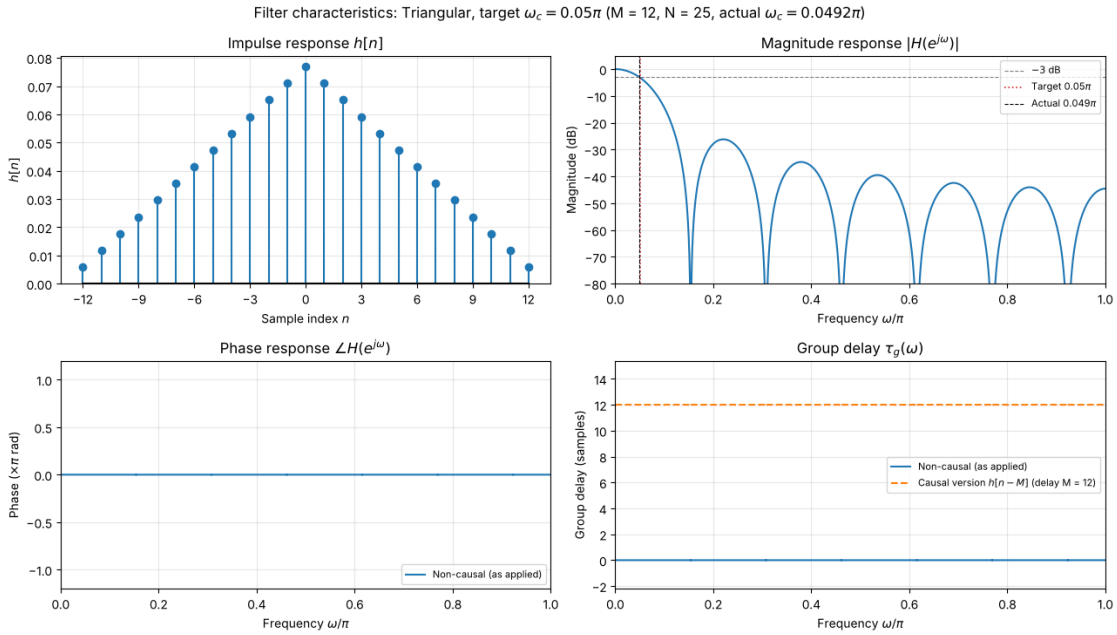

Filter characteristics: Rectangular, target $\omega_c = 0.2\pi$ (M = 2, N = 5, actual $\omega_c = 0.1803\pi$)
Impulse response $h[n]$
$h[n]$
Sample index n
Magnitude response $|H(e^{j\omega})|$
Magnitude (dB)
−3 dB
Target 0.2π
Actual 0.180π
Frequency ω/π
Phase response $\angle H(e^{j\omega})$
Phase (×π rad)
Non-causal (as applied)
Frequency ω/π
Group delay $\tau_g(\omega)$
Group delay (samples)
Non-causal (as applied)
Causal version $h[n - M]$ (delay M = 2)
Frequency ω/π

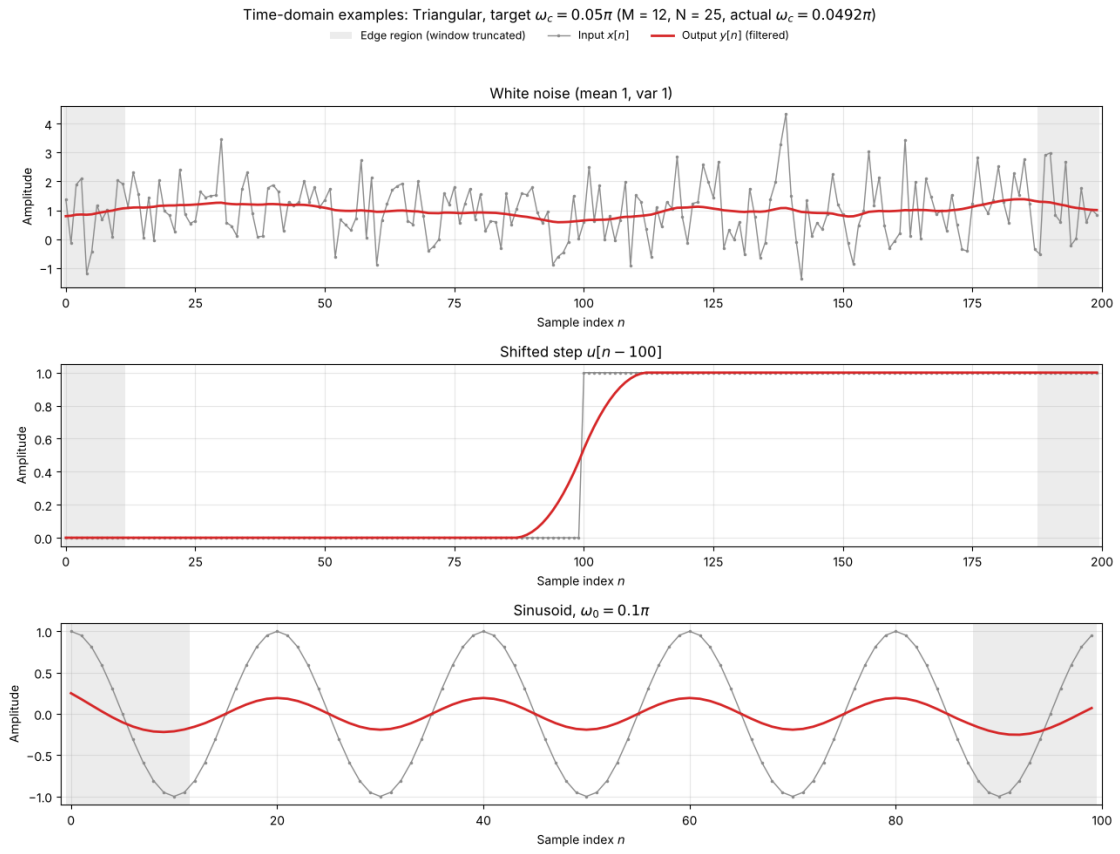

Time-domain examples: Rectangular, target $\omega_c = 0.2\pi$ (M = 2, N = 5, actual $\omega_c = 0.1803\pi$)
Edge region (window truncated)
Input $x[n]$
Output $y[n]$ (filtered)
White noise (mean 1, var 1)
Amplitude
Sample index n
Shifted step $u[n - 100]$
Amplitude
Sample index n
Sinusoid, $\omega_0 = 0.1\pi$
Amplitude
Sample index n

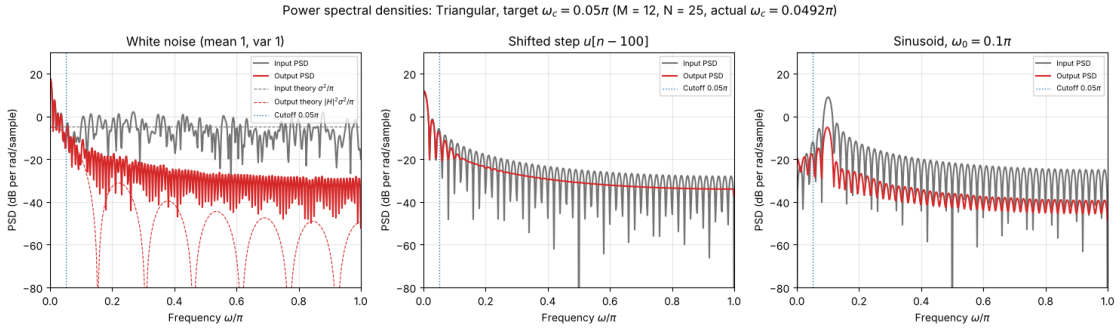


Triangular (Bartlett)

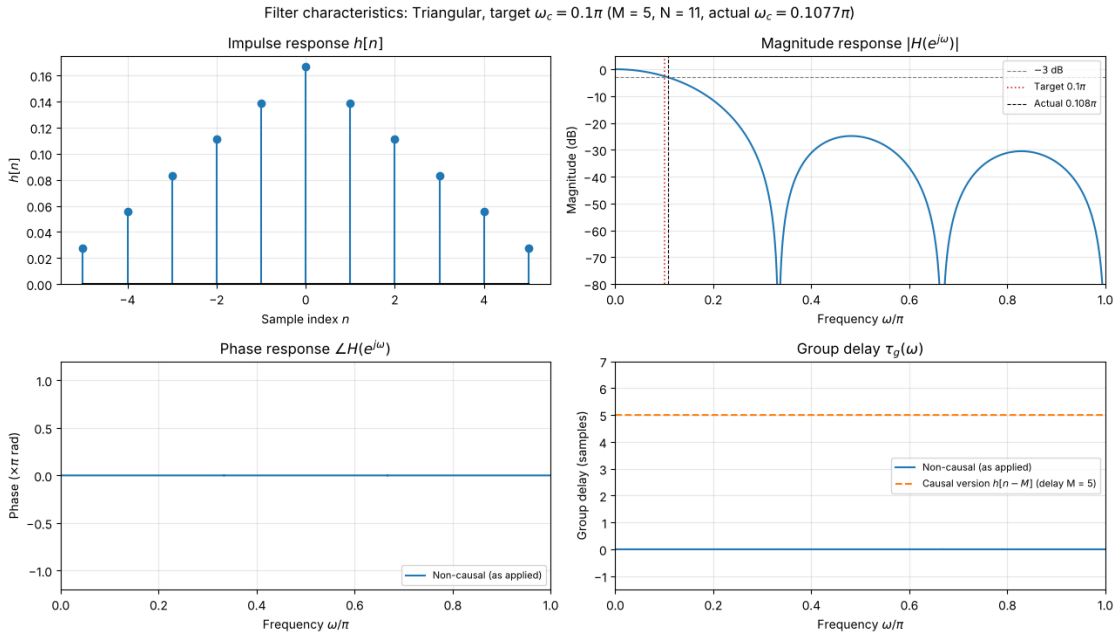
Triangular, $\omega_c = 0.05\pi$

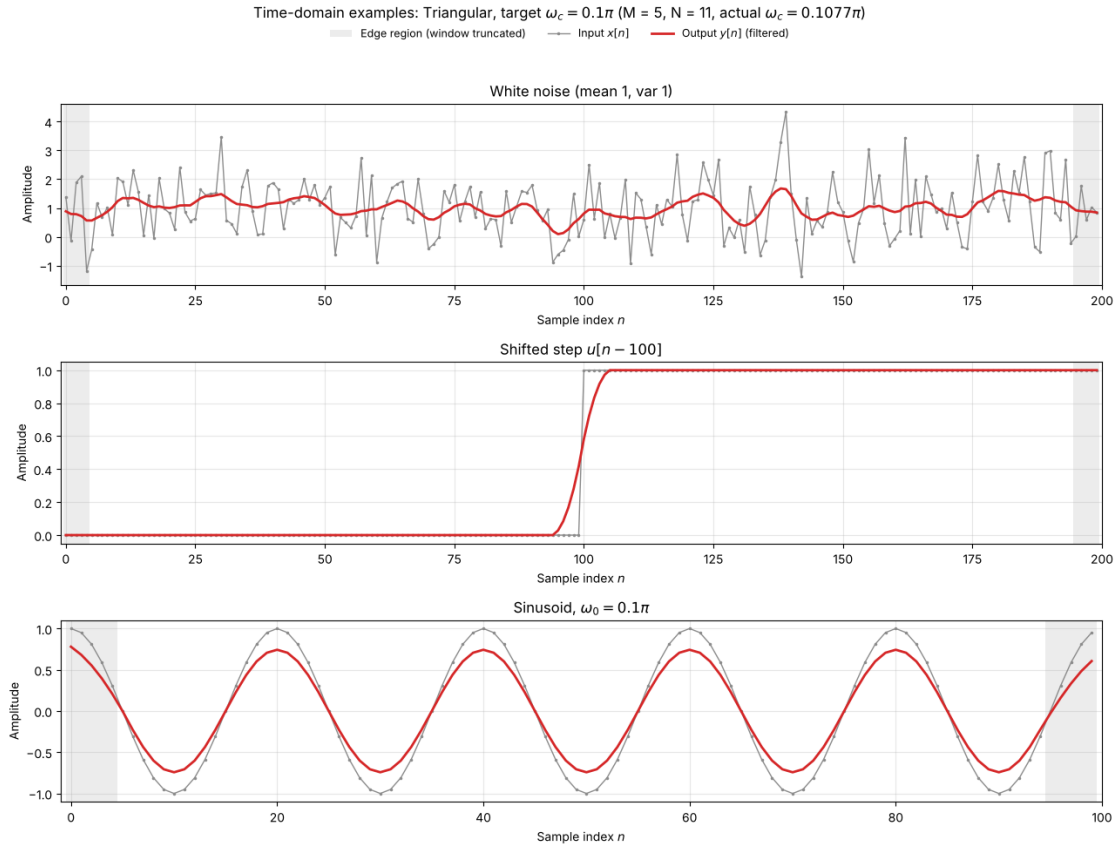


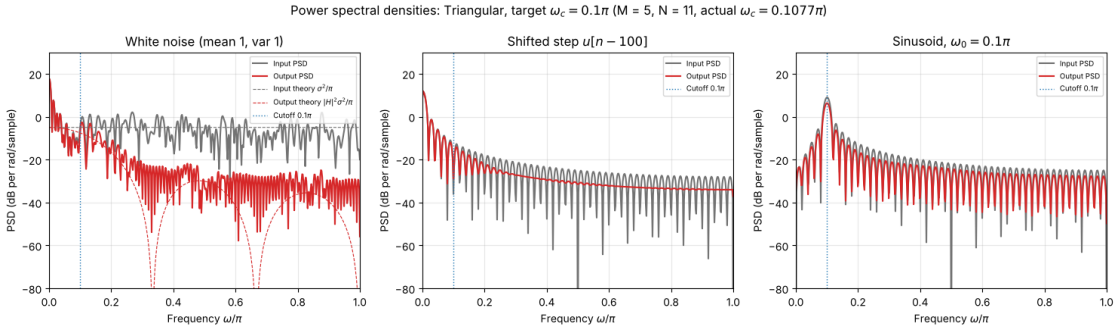




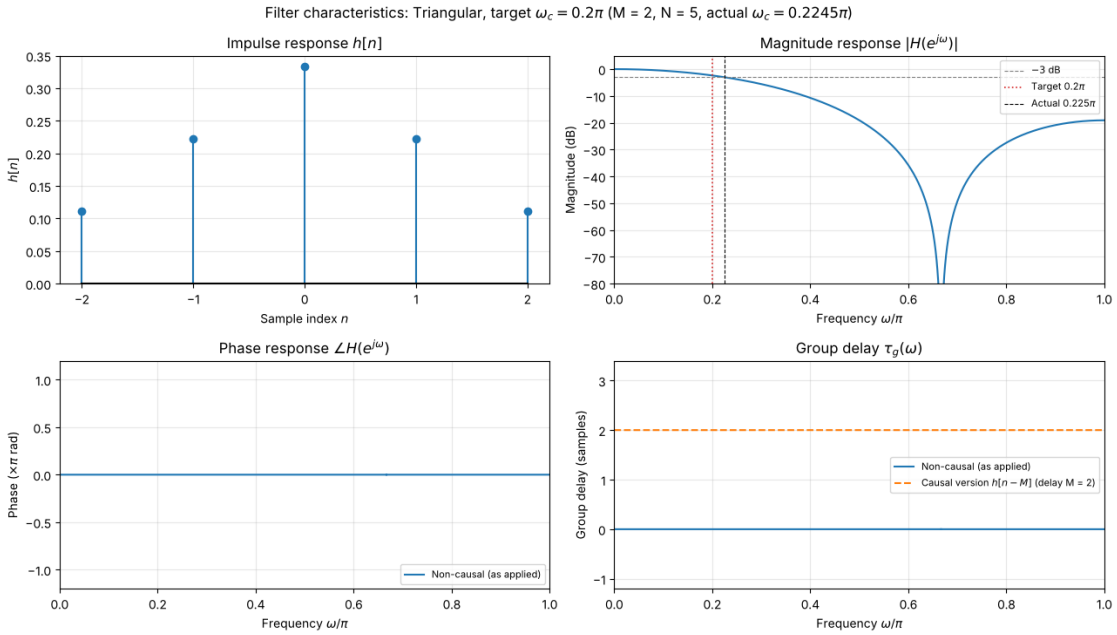
Triangular, $\omega_c = 0.1\pi$

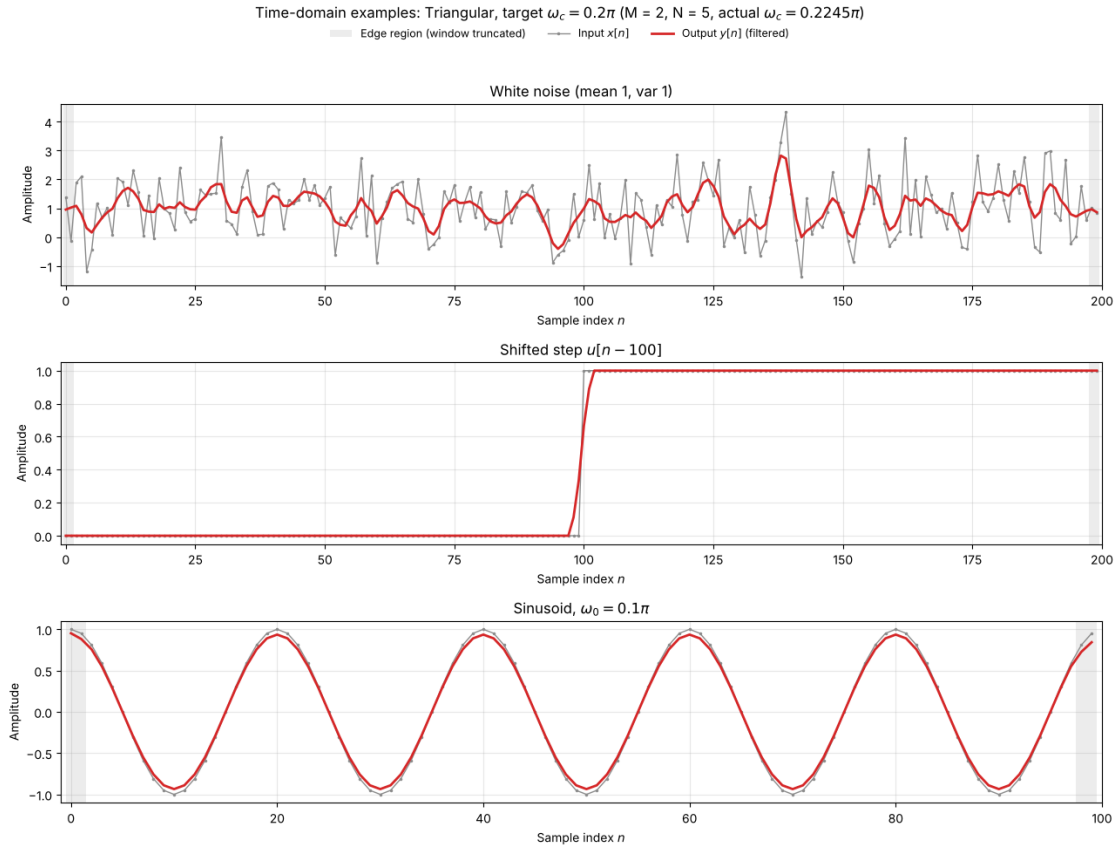

Filter characteristics: Triangular, target $\omega_c = 0.1\pi$ (M = 5, N = 11, actual $\omega_c = 0.1077\pi$)
Impulse response $h[n]$
$h[n]$
Sample index n
Magnitude response $|H(e^{j\omega})|$
Magnitude (dB)
−3 dB
Target 0.1π
Actual 0.108π
Frequency ω/π
Phase response $\angle H(e^{j\omega})$
Phase (×π rad)
Non-causal (as applied)
Frequency ω/π
Group delay $\tau_g(\omega)$
Group delay (samples)
Non-causal (as applied)
Causal version $h[n-M]$ (delay M = 5)
Frequency ω/π

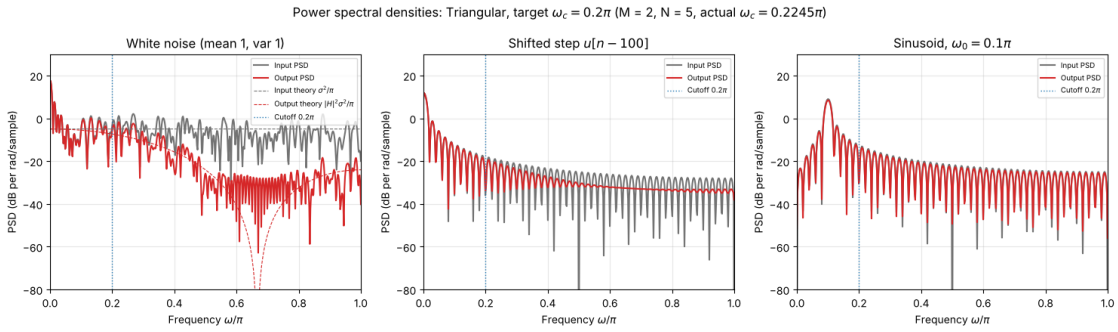

Time-domain examples: Triangular, target $\omega_c = 0.1\pi$ (M = 5, N = 11, actual $\omega_c = 0.1077\pi$)
Edge region (window truncated)
Input $x[n]$
Output $y[n]$ (filtered)
White noise (mean 1, var 1)
Amplitude
Sample index n
Shifted step $u[n-100]$
Amplitude
Sample index n
Sinusoid, $\omega_0 = 0.1\pi$
Amplitude
Sample index n



Triangular, $\omega_c = 0.2\pi$

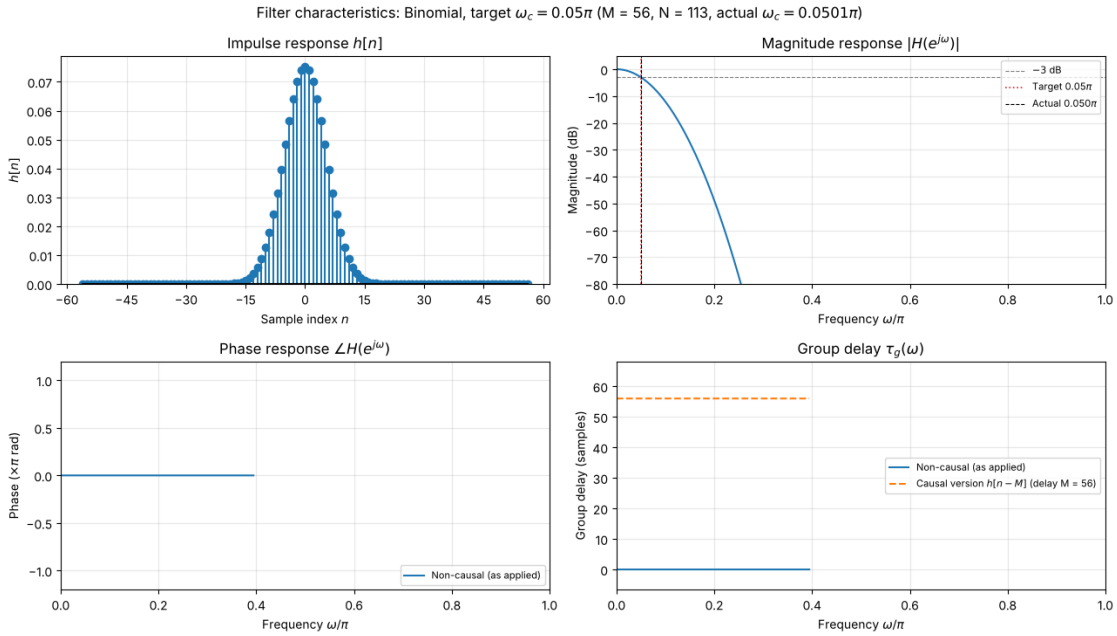


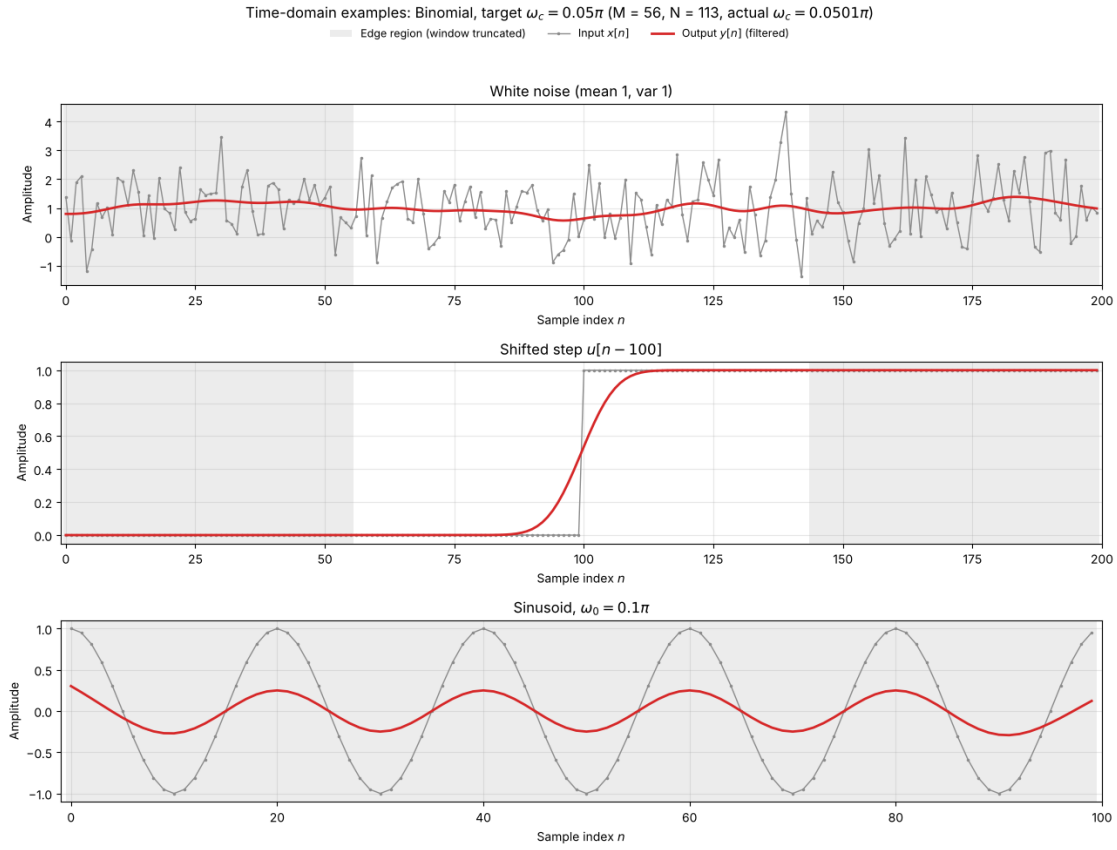


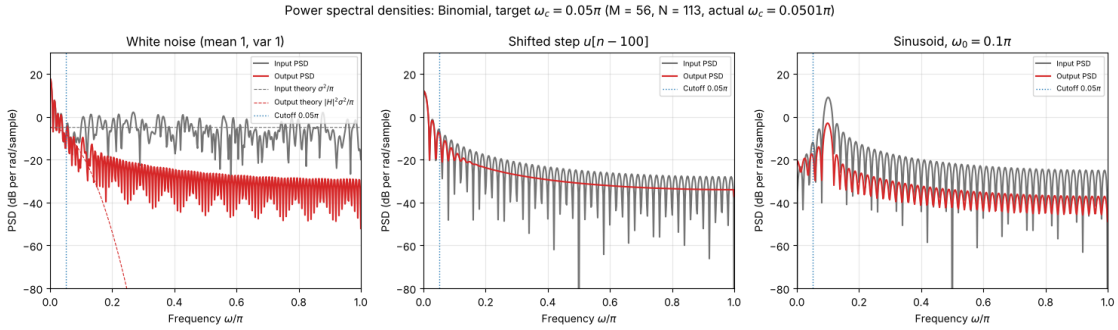


Binomial

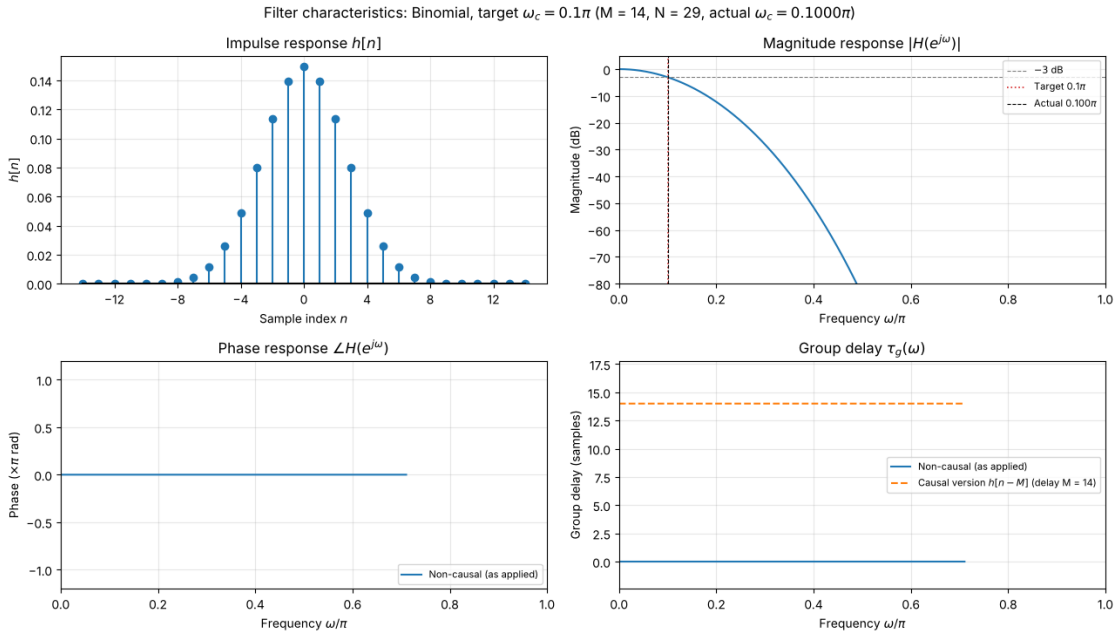
Binomial, $\omega_c = 0.05\pi$

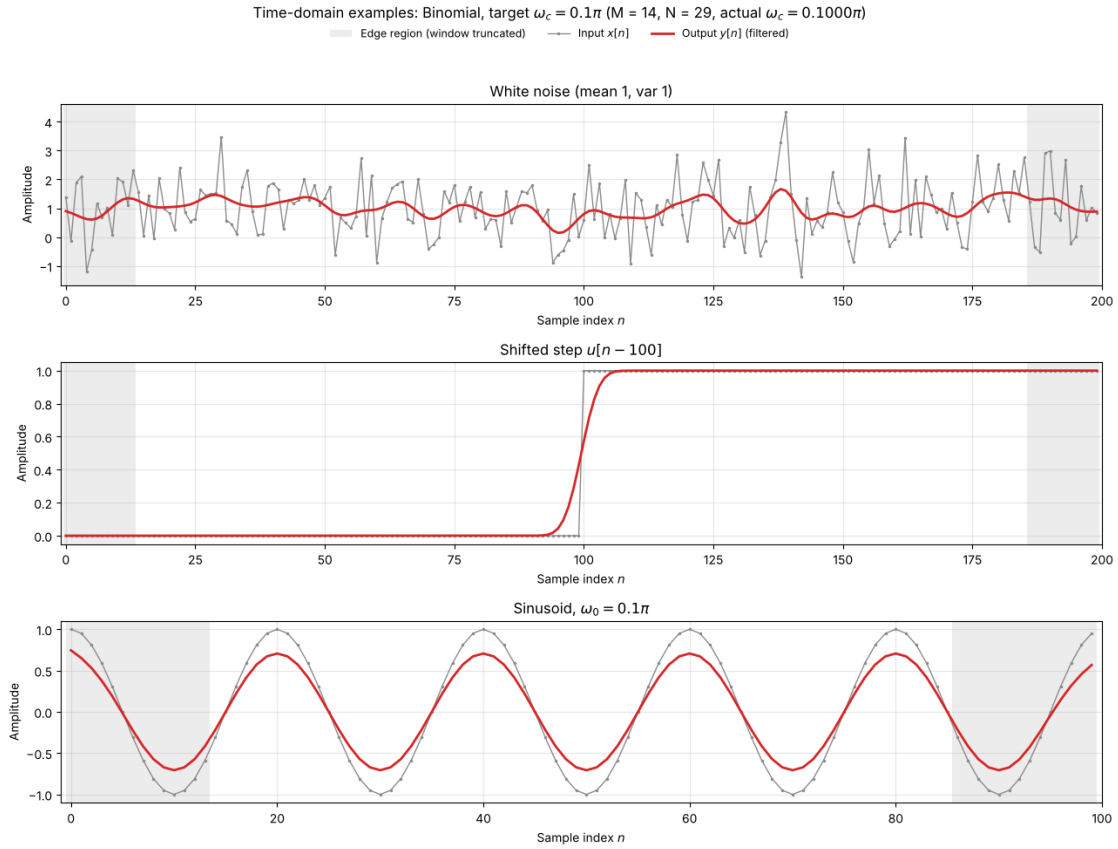

Filter characteristics: Binomial, target $\omega_c = 0.05\pi$ (M = 56, N = 113, actual $\omega_c = 0.0501\pi$)
Impulse response $h[n]$
$h[n]$
Sample index n
Magnitude response $|H(e^{j\omega})|$
Magnitude (dB)
−3 dB
Target 0.05π
Actual 0.050π
Frequency ω/π
Phase response $\angle H(e^{j\omega})$
Phase (×π rad)
Non-causal (as applied)
Frequency ω/π
Group delay $\tau_g(\omega)$
Group delay (samples)
Non-causal (as applied)
Causal version $h[n - M]$ (delay M = 56)
Frequency ω/π

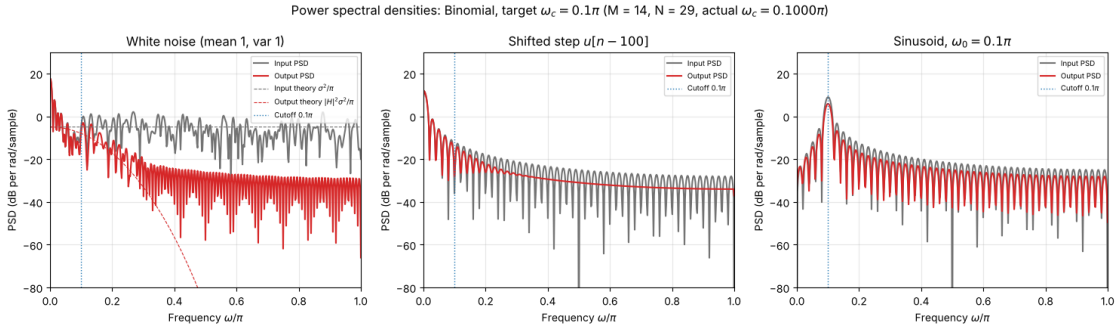

Time-domain examples: Binomial, target $\omega_c = 0.05\pi$ (M = 56, N = 113, actual $\omega_c = 0.0501\pi$)
Edge region (window truncated)
Input $x[n]$
Output $y[n]$ (filtered)
White noise (mean 1, var 1)
Amplitude
Sample index n
Shifted step $u[n - 100]$
Amplitude
Sample index n
Sinusoid, $\omega_0 = 0.1\pi$
Amplitude
Sample index n



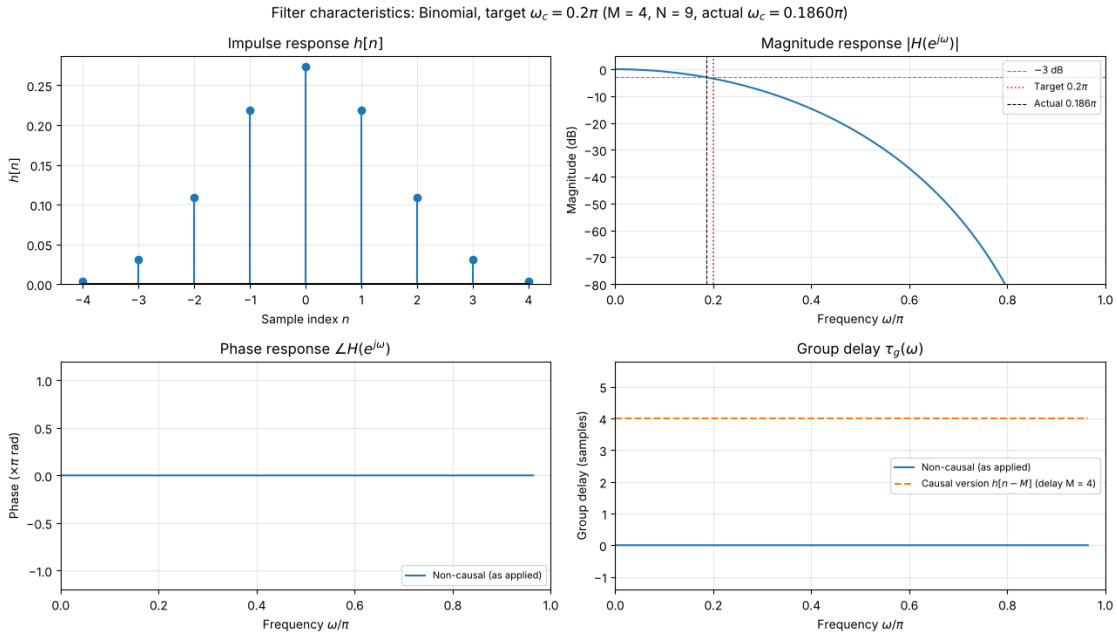
Binomial, $\omega_c = 0.1\pi$

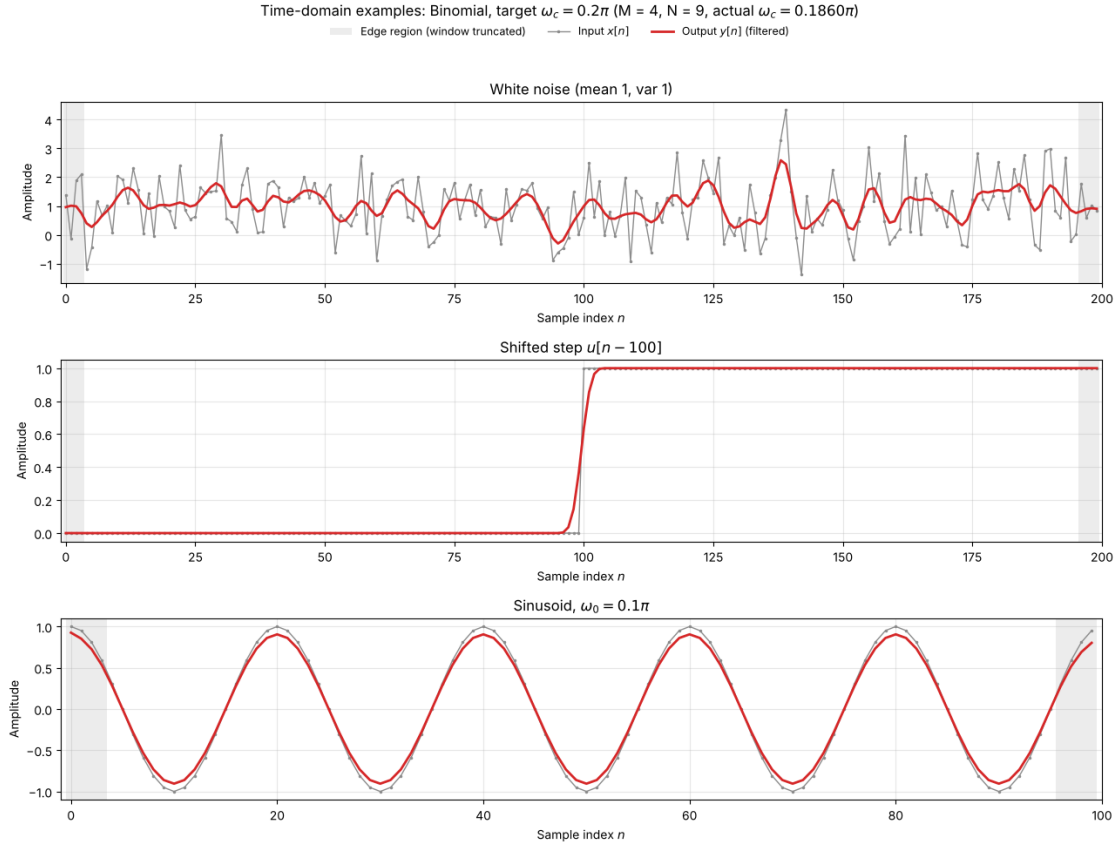


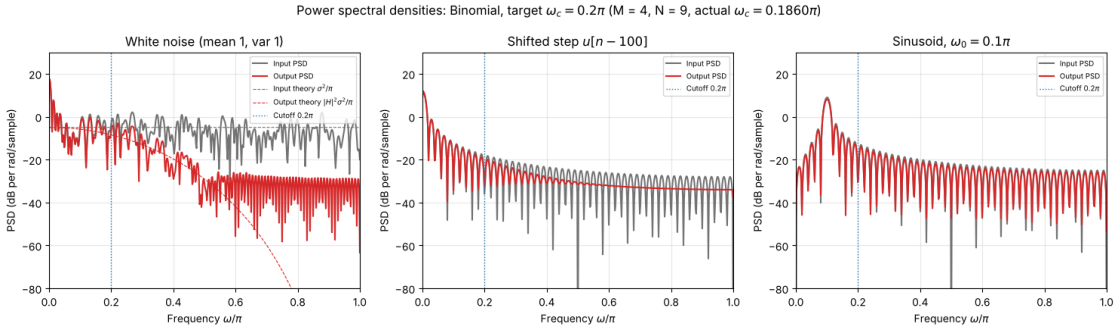




Binomial, $\omega_c = 0.2\pi$


Filter characteristics: Binomial, target $\omega_c = 0.2\pi$ (M = 4, N = 9, actual $\omega_c = 0.1860\pi$)
Impulse response $h[n]$
$h[n]$
Sample index n
Magnitude response $|H(e^{j\omega})|$
Magnitude (dB)
−3 dB
Target 0.2π
Actual 0.186π
Frequency ω/π
Phase response $\angle H(e^{j\omega})$
Phase (×π rad)
Non-causal (as applied)
Frequency ω/π
Group delay $\tau_g(\omega)$
Group delay (samples)
Non-causal (as applied)
Causal version $h[n-M]$ (delay M = 4)
Frequency ω/π


Time-domain examples: Binomial, target $\omega_c = 0.2\pi$ (M = 4, N = 9, actual $\omega_c = 0.1860\pi$)
Edge region (window truncated)
Input $x[n]$
Output $y[n]$ (filtered)
White noise (mean 1, var 1)
Amplitude
Sample index n
Shifted step $u[n-100]$
Amplitude
Sample index n
Sinusoid, $\omega_0 = 0.1\pi$
Amplitude
Sample index n



Problem 3: Explain how this type of filter might be used as a highpass filter. Provide an example with the following:

- Magnitude response
- Phase response
- Group delay
- Impulse response
- Time domain example of the filter applied to a white noise, signal with 200 samples
- Power spectral density plot of the input signal
  - ***Please use the periodogram method or an appropriately scaled FFT to compute the PSD on all signals.***
- Power spectral density plot of the output signal
  - ***Please use the periodogram method or an appropriately scaled FFT to compute the PSD on all signals.***

Idea: subtract the smoothed signal from the original

A smoothing filter keeps the slowly varying part of a signal (the trend). Subtracting that trend from the original leaves only the fast-varying part. In filter terms, the highpass filter is the **complement** of the lowpass filter:

$$y_{\text{hp}}[n] = x[n] - (h_{\text{lp}} * x)[n] \quad \Longleftrightarrow \quad h_{\text{hp}}[n] = \delta[n] - h_{\text{lp}}[n] \quad \Longleftrightarrow \quad H_{\text{hp}}(e^{j\omega}) = 1 - H_{\text{lp}}(e^{j\omega}).$$

This is a standard construction, which Smith calls **spectral inversion** [1]. The lowpass filter and an "all-pass" path, whose impulse response is $\delta[n]$, run in parallel, and the lowpass output is subtracted from the original. Equivalently, flipping the sign of every lowpass tap and adding 1 to the center tap gives the highpass kernel, and its frequency response is the lowpass response flipped top for bottom [1]. Image processing uses the same idea to sharpen images [2].

Properties inherited from the lowpass filter:

- **Zero DC gain:** $H_{\text{hp}}(e^{j0}) = 1 - \sum_n h_{\text{lp}}[n] = 1 - 1 = 0$. The unit-DC-gain normalization from Problem 1 is exactly what makes the highpass reject DC completely, so any constant offset (such as the mean of 1 in the test noise) is removed. Equivalently, the highpass taps sum to zero [2].
- **Even symmetry and zero phase:** $\delta[n]$ and $h_{\text{lp}}[n]$ are both even, so $h_{\text{hp}}[n]$ is even and $H_{\text{hp}}(e^{j\omega})$ is real [3]. Because $H_{\text{lp}}(e^{j\omega}) \leq 1$ for every non-negative filter (the taps sum to 1, so

$|H| \le \sum |h| = 1$), $H_{\text{hp}}(e^{j\omega}) = 1 - H_{\text{lp}}(e^{j\omega}) \ge 0$ at every frequency. The phase is therefore zero wherever $H_{\text{hp}} \neq 0$, and the group delay is zero (or a constant M samples for the causal version) [3].

- **Same length:** h_{hp} has the same $2M+1$ taps as h_{lp}. Only the center tap changes, to $1-h_{\text{lp}}[0]$ [1].
- **Not non-negative:** $h_{\text{hp}}[0] = 1 - h_{\text{lp}}[0] > 0$, and every other tap is $-h_{\text{lp}}[n] \le 0$. The taps sum to 0, as a filter with zero DC gain must.

Cutoff. For the highpass filter, the -3 dB cutoff is the frequency where H_{hp} *rises* to $1/\sqrt{2}$, which is where the lowpass filter has fallen to $1 - 1/\sqrt{2} \approx 0.293$ (-10.7 dB). So a highpass cutoff of ω_c needs a lowpass filter whose own -3 dB cutoff is below ω_c, and therefore a somewhat longer filter.

Edge handling. I compute the output as $y_{\text{hp}} = x - \text{apply_filter}(x, h_{\text{lp}})$, using the same normalized convolution as in Problem 2. Because the lowpass part preserves DC exactly at the edges, the highpass output still rejects DC at the edges, so the output does not jump at the ends of the record. (Zero-padding would make the smoothed trend sag toward 0 at the ends, and the highpass output would then show a spurious jump of about 1 at each end of the mean-1 noise.)

References

[1] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, 2nd ed. San Diego, CA: California Technical Publishing, 1999, ch. 14, "High-Pass, Band-Pass and Band-Reject Filters," Figs. 14-5 and 14-6. [Online]. Available: https://www.dspguide.com/ch14/5.htm

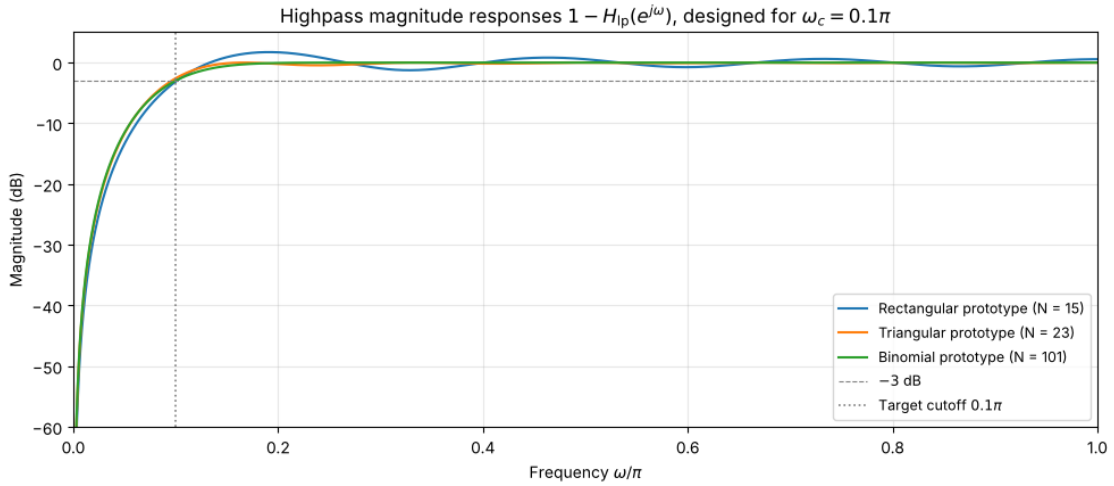
[2] R. Wang, "Sharpening," E161 lecture notes, Harvey Mudd College. [Online]. Available: https://pages.hmc.edu/ruye/e161/lectures/gradient/node1.html

[3] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2009, Sec. 5.7 (systems with generalized linear phase).

Choosing the lowpass prototype

For a target highpass cutoff of $\omega_c = 0.1\pi$, I find the length of each Problem 1 filter whose complement comes closest to that cutoff, and compare the three highpass magnitude responses.

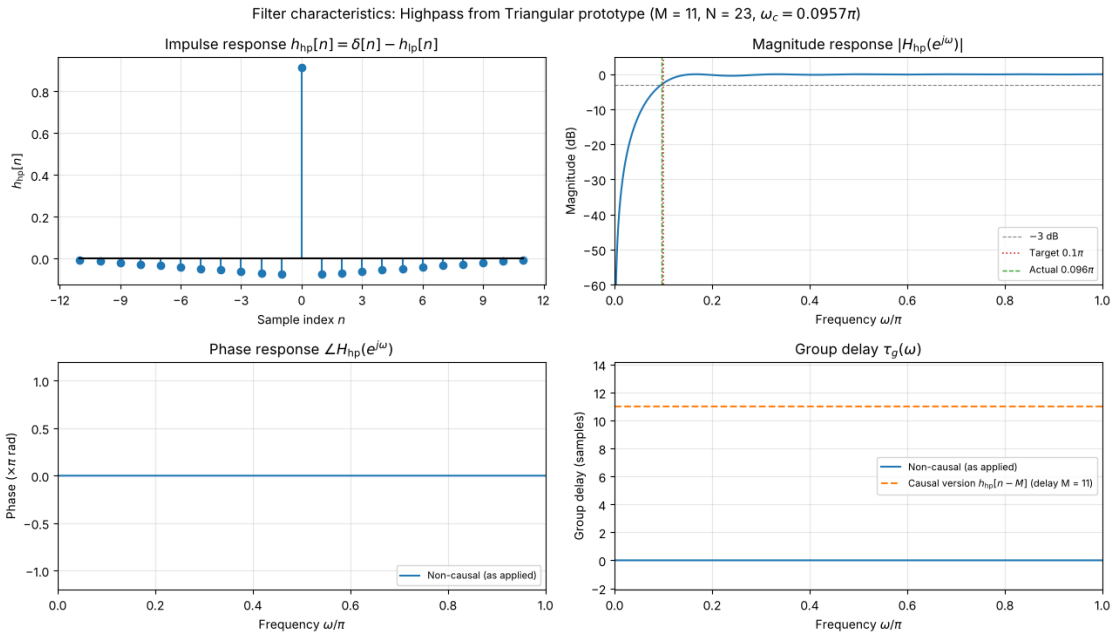
```
Prototype       M    N  HP cutoff  Max gain
Rectangular     7   15    0.1008pi    +1.73 dB
Triangular     11   23    0.0957pi    +0.00 dB
Binomial       50  101    0.0996pi    +0.00 dB
```

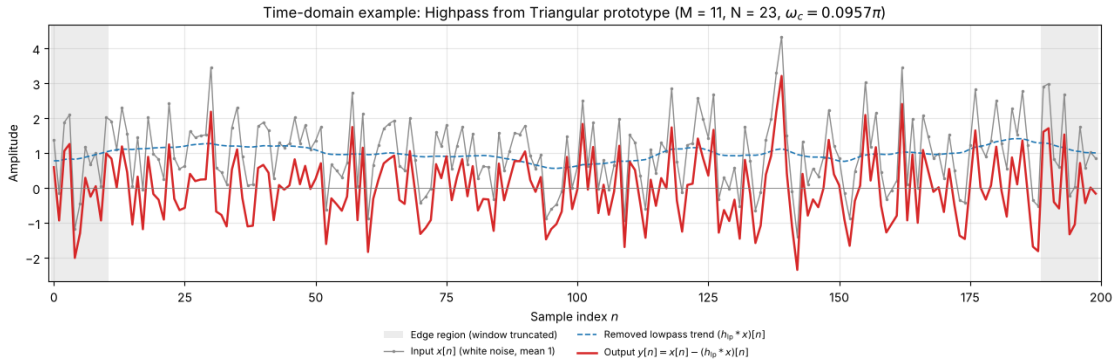


Choice: triangular prototype. The three complements behave differently above the cutoff:

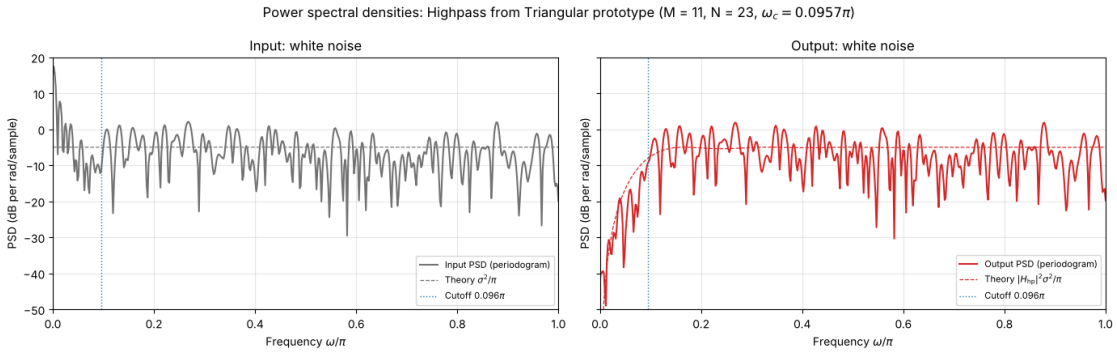
- **Rectangular:** its lowpass sidelobes go negative, so $1 - H_{\text{lp}}$ rises *above* 1. The passband ripples, amplifying some frequencies by up to about +1.7 dB.
- **Triangular:** $H_{\text{lp}}(e^{j\omega}) \geq 0$ (a squared Dirichlet kernel), so $0 \leq H_{\text{hp}} \leq 1$. The passband never exceeds unity gain, ripple is small, and the filter is short.
- **Binomial:** the cleanest response (monotonic, no ripple at all), but its gradual roll-off means it needs about 4× as many taps for the same cutoff. That makes the edge regions half of a 200-sample record.

The triangular prototype is the best balance of passband flatness and length here, so I use it for the detailed example below.






```
Input : mean = +1.000, variance = 1.000
Output: mean = -0.001, variance = 0.918 (theory for white input: input variance
x sum(h_hp^2) = 0.889)
```



Observations.

- The input's mean of 1 shows up as the large peak at $\omega = 0$ in the input PSD. The highpass output has a mean near 0, and its PSD drops steeply toward DC, confirming $H_{\text{hp}}(e^{j0}) = 0$.
- Above the cutoff the output PSD matches the input, since $H_{\text{hp}} \approx 1$ there. The filter removes the slow wander and passes the sample-to-sample fluctuations unchanged.
- The output variance is slightly below the input variance, because only the low-frequency band, a small fraction of the total power, is removed.
- The phase is zero, so the output is aligned in time with the input. A spike in the input appears at the same index in the output.

Problem 4: Explain what you think the best way is to handle edge conditions with this filter? Provide good, defensible rationale for your proposal. There is no single right answer.

The problem

A non-causal filter of length $2M + 1$ needs M input samples on **each side** of the output sample. For the first and last M outputs, some of those samples don't exist. Any edge method is really an assumption about the missing samples, or a decision not to make one.

Candidate methods

Method	What it assumes about the missing samples
Zero padding	They are 0
Edge replication	They equal the end sample, $x[0]$ or $x[L-1]$
Even reflection	The signal mirrors about the end sample: $x[-k] = x[k]$
Odd reflection	The signal continues its local trend: $x[-k] = 2x[0] - x[k]$
Periodic extension	The signal repeats: $x[-k] = x[L-k]$
Normalized convolution	Nothing. It uses only the samples that exist and rescales the truncated filter so its taps sum to 1
Discard ("valid")	Nothing. It drops the M outputs at each end

Normalized convolution (used in Problems 2 and 3) computes

$$y[n] = \frac{\sum_k h[k]\, x[n-k]}{\sum_k h[k]}, \qquad \text{both sums over } k \text{ with } 0 \le n-k \le L-1,$$

which amounts to a different, truncated-and-renormalized filter at each edge sample. Each output is still a weighted average of real samples with weights summing to 1.

How I compare them

I use the triangular filter designed for $\omega_c = 0.05\pi$ ($M = 12$, $N = 25$), long enough for edge effects to be clearly visible, and three tests:

1. **The shifted step** $u[n-100]$ from Problem 2, a signal whose two ends have different levels.

- **What it probes:** methods that make an assumption about the far end of the signal. The step is 0 at the left end and 1 at the right end. A method that's fine at one end can fail at the other.

2. **Monte Carlo on mean-1, variance-1 white noise** (4000 realizations). The ideal smoothed output is the constant 1, so I measure **bias** (does the method pull the estimate away from the true level?) and **RMSE** (bias and noise combined) at each distance from the edge.

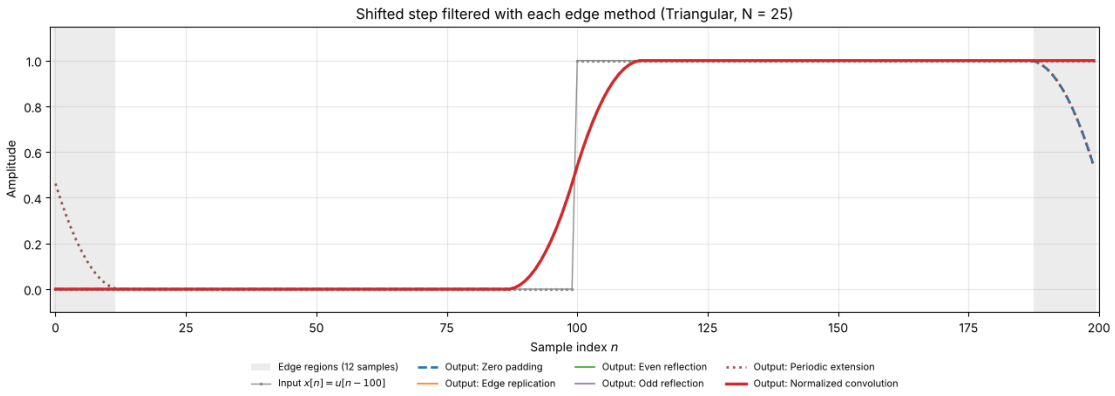
- **What it probes:** This test exposes a cost that the step and ramp tests can't, because those signals have no noise. Some methods reuse a single noisy sample many times

3. **A noise-free ramp** $x[n] = 0.02\,n$. A symmetric filter with unit DC gain reproduces a straight line exactly in the interior, so any output error comes from the edge method alone. This tests how each method handles a trend at the boundary.

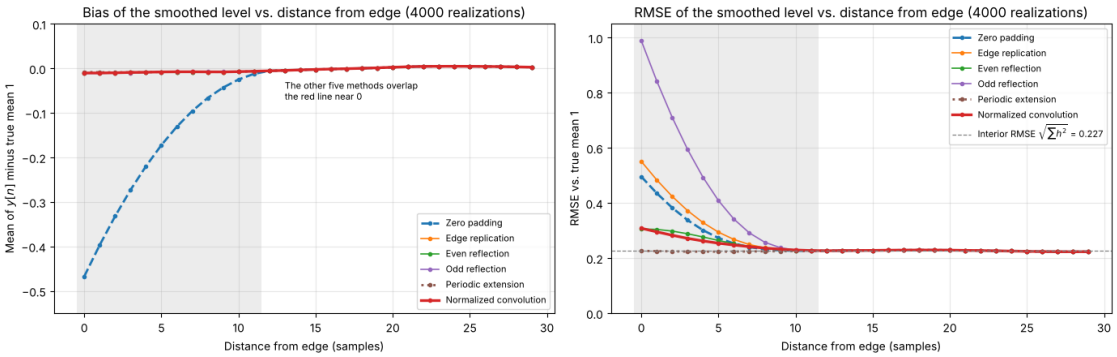
- **What it probes:** how each method handles a trend at the boundary, with nothing else mixed in.

`Comparison filter: Triangular, M = 12, N = 25`

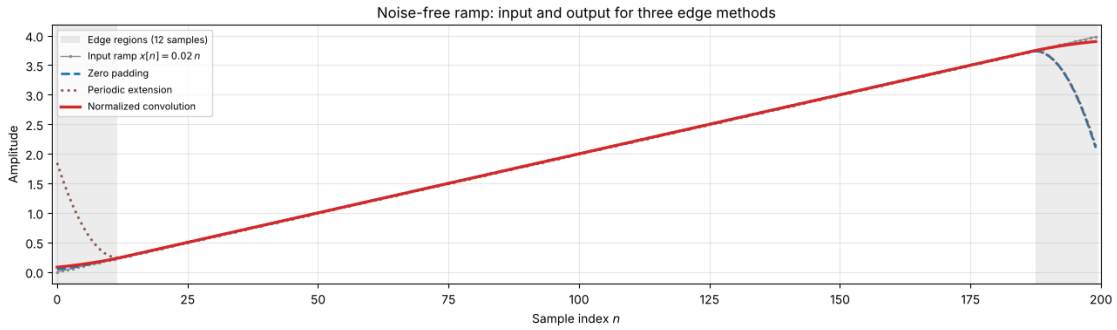
Test 1: Shifted step

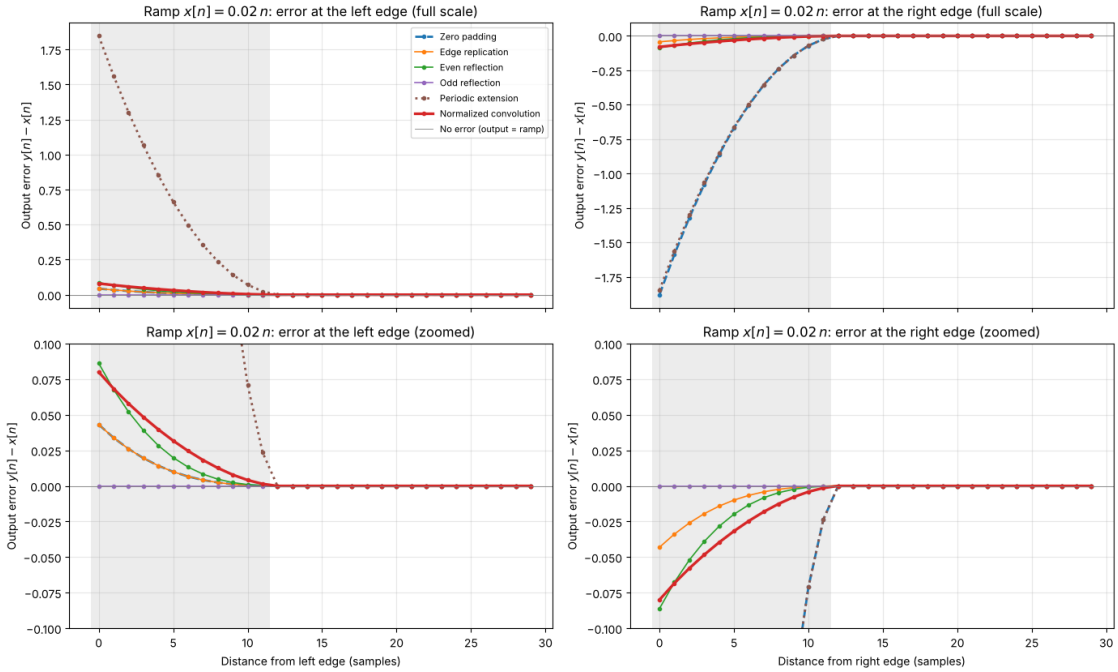


Test 2: Monte Carlo on mean-1 white noise



Test 3: Noise-free ramp





Summary table

Method	Bias at n=0	RMSE at n=0	Ramp error left end	Ramp error right end
Zero padding	-0.467	0.496	+0.043	-1.880
Edge replication	-0.011	0.551	+0.043	-0.043
Even reflection	-0.010	0.306	+0.086	-0.086
Odd reflection	-0.011	0.989	+0.000	+0.000
Periodic extension	-0.008	0.226	+1.846	-1.846
Normalized convolution	-0.010	0.308	+0.080	-0.080

```
Interior RMSE for reference: 0.227
```

My recommendation: normalized convolution, with the edge samples flagged

What the tests show (values at the edge sample, $n = 0$ or $n = L - 1$)

Method	Bias on mean-1 noise	RMSE on noise (interior: 0.23)	Ramp error	Verdict
Zero padding	-0.47	0.50	-1.88 (right end)	Worst: pulls the output toward 0
Edge replication	≈ 0	0.55	± 0.04	Noisy: leans on one sample, $x[0]$
Odd reflection	≈ 0	0.99	0	No smoothing at the endpoint: $y[0] = x[0]$
Periodic extension	≈ 0	0.23	± 1.85	Fails whenever the two ends differ
Even reflection	≈ 0	0.31	± 0.09	Good
Normalized convolution	≈ 0	0.31	± 0.08	**Good (my choice)**

No method wins every test. Even reflection and normalized convolution tie for the best overall balance.

Why normalized convolution over even reflection

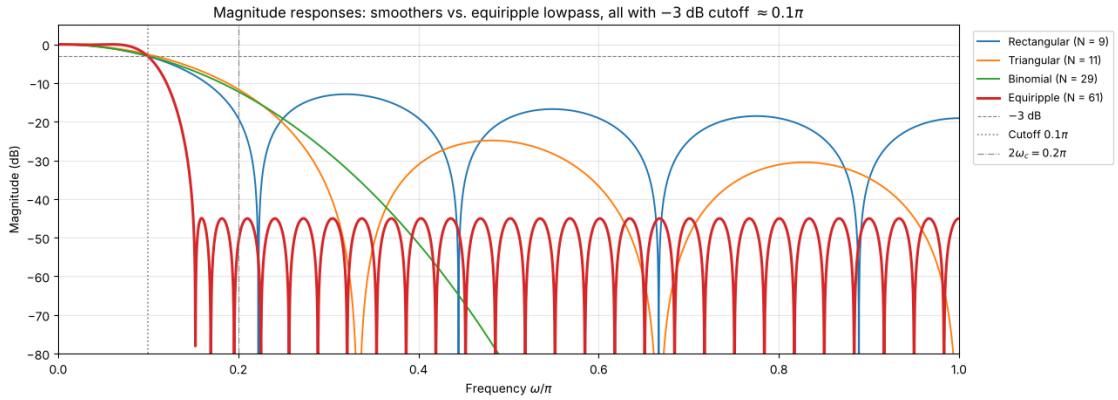
1. **It uses only real data.** Every output is a weighted average of samples that actually exist. Even reflection quietly reuses samples.
2. **The DC gain is exactly 1 everywhere, and the output stays within the range of the input.** Non-negative data can never produce a negative smoothed value.
3. **It still works when the filter is longer than the signal** (the 113-tap binomial filter on the 100-sample sinusoid) **or when the data has gaps.**
4. **The highpass filter inherits it.** In Problem 3, $x -$ (smoothed x) keeps zero DC gain right up to the edges.

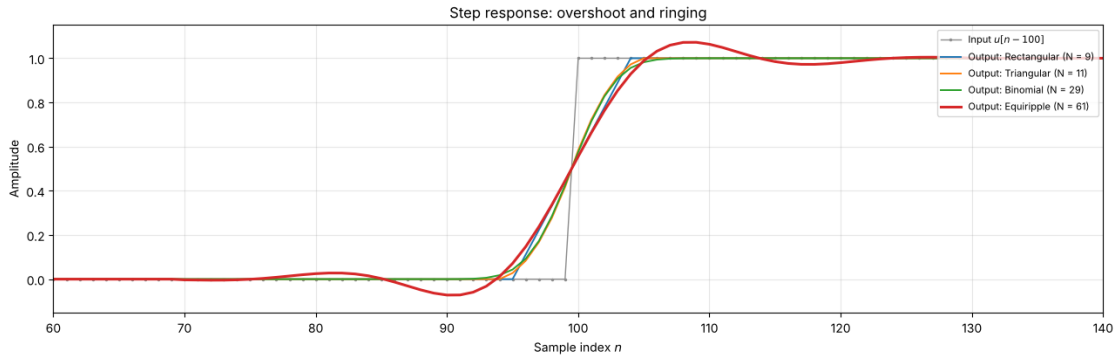
This only works because $h[n] \geq 0$**.** The denominator $\sum h[k]$ is then always positive. For a filter with negative taps, the sum of the remaining taps can approach zero or change sign, and the method breaks down.

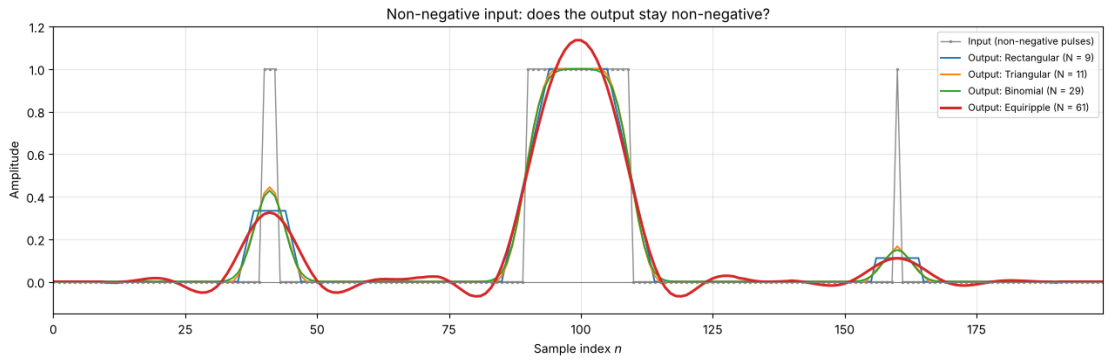
Problem 5: What type of circumstances would you want to apply this type of filter, rather than a well-designed frequency selective filter? What are the design tradeoffs?

To make the tradeoffs concrete, I compare the Problem 2 smoothers designed for $\omega_c = 0.1\pi$ with an **equiripple (Parks-McClellan) lowpass filter** designed for the same -3 dB cutoff: passband edge 0.07π, stopband edge 0.15π, $N = 61$ taps. The equiripple design is optimal in the minimax sense, so it represents the best a frequency-selective FIR design can do at that length.

The time-domain tests use zero-phase (centered) filtering with edge replication for **every** filter, so the comparison is fair. (Normalized convolution can't be used for the equiripple filter because its taps go negative; see Problem 4.)







```
Filter                    -3 dB  Gain at  Atten. at       Step  Min output,
Arithmetic per output
                         cutoff    0.05pi  0.2pi (dB)  overshoot  pulse input
Rectangular (N = 9)      0.099pi    0.920      -12.9       0.0%       +0.000  2
adds, 1 scale (running sum)
Triangular (N = 11)      0.108pi    0.930      -11.6       0.0%       +0.000  4
adds, 1 scale (two running sums)
Binomial (N = 29)        0.100pi    0.917      -12.2       0.0%       +0.000  28
adds, 1 shift ([1,1] cascade)
Equiripple (N = 61)      0.099pi    0.996      -45.0       7.2%       -0.069  31
mults, 60 adds (symmetric FIR)
```

When to use a non-negative smoother instead of a frequency-selective filter

What the comparison shows

- **Frequency selectivity: the equiripple filter wins.** At 0.2π it attenuates by about 45 dB, while the smoothers manage only about 12 dB. The smoothers also droop inside the passband (gain of about 0.92 at 0.05π, versus 1.0 for equiripple).
- **Time-domain behavior: the smoothers win.** Their step response has **no overshoot**, and a non-negative input never produces a negative output. The equiripple filter overshoots by about 7% and rings at every edge.
- **Cost: the smoothers win.** A moving average is a running sum and needs essentially no multiplies. The equiripple filter needs 31 multiplies per output.

Use a non-negative smoother when:

1. **The time-domain shape matters:** edges, peaks, thresholds, trends. Ringing would create false peaks or false threshold crossings.
2. **The signal is physically non-negative:** counts, intensities, pixel values, power.
3. **Computation is limited:** embedded systems or FPGAs. CIC decimators are cascaded moving averages for this reason.

Use a frequency-selective filter when:

1. **Two components are close in frequency** and must be separated (for example, removing a 60 Hz line).
2. **There is a stopband specification** (for example, -60 dB anti-aliasing before decimation).
3. **The passband must be flat** for amplitude accuracy.

Summary

	Non-negative smoother	Frequency-selective (equiripple)
Transition band	Wide	Narrow
Stopband attenuation	Poor (-13 to -26 dB sidelobes)	High, set by the design spec
Step response	No overshoot	Overshoot and ringing ($\approx 7\%$)
Output range	Always within the input range	Can exceed it or go negative

	Non-negative smoother	Frequency-selective (equiripple)
Design effort	One parameter (length)	Band edges, ripple, weights
Cost	Nearly multiplier-free	One multiply per pair of taps

Project

THE FOLLOWING APPLICATION OF A NON-NEGATIVE FIR FILTER WAS DONE WITH AI

Problem 7: What is the problem?

The problem

From video of a transmission span, estimate each conductor's galloping mode, frequency, and peak-to-peak amplitude, and raise an alert **quickly and without false alarms**. Each span station gives a vertical-displacement signal sampled at 30 fps:

$$x[n] = \underbrace{\text{gallop}}_{\text{signal}} + \underbrace{b[n]}_{\text{blowout}} + \underbrace{s[n]}_{\text{camera shake}} + \underbrace{v[n]}_{\text{noise}}$$

What makes this hard:

- **Gallop and blowout are close in frequency.** The gallop modes sit at about 0.25, 0.50, and 0.76 Hz, and blowout sits just below 0.25 Hz.
- **The measurement is coarse:** about 0.21 m per pixel at default.
- **The alert must be fast** ($\leq$ 30 s), yet never trigger on blowout or ice shedding.

Where non-negative FIR filters fit

Following Problem 5, the project question becomes: **where is a simple smoother good enough, and what do I give up?**

1. Blowout rejection: subtract the smoothed signal (the Problem 3 highpass). The gallop modes are harmonics: $f_n = nf_1$. A moving average exactly one gallop period long (≈ 3.9 s) has zeros at $f_1, 2f_1, 3f_1, \ldots$. It therefore captures the blowout but none of the gallop, and subtracting it removes the blowout while leaving every gallop mode untouched. Applying the subtraction twice, $(1 - H)^2$, makes the blowout rejection stronger.

	Kaiser FIR (proposal)	Moving-average complement
Blowout attenuation at 0.05 Hz	$\geq$ 40 dB	48.8 dB
Gain at the gallop modes	$\pm$0.5 dB	$\pm$0.5 dB
Gain between modes	flat	up to +3.4 dB
Delay	$\approx$ 11 s	$\approx$ 3.7 s
Arithmetic	167-tap convolution	two running sums

Trade-off: the passband is not flat, but that's acceptable because gallop energy only sits at the harmonics. In exchange, the delay drops by about 3×, which directly helps alert speed. The catch is that the moving-average length depends on knowing f_1; a 5% error in f_1 costs about 1.4 dB.

2. Smoothing the amplitude for the alert. Square the high-pass output and average it over one gallop period with the same moving average. This gives a ripple-free power estimate, and the amplitude follows from it ($\sqrt{2}\times$ RMS for one mode). Because the taps are non-negative:

- the estimate never overshoots, so a sudden transient such as ice shedding can't briefly spike past the alert threshold;
- the delay is about 4 s, versus 16 s for the STFT.

3. Smoothing the PSD without segmenting. Instead of Welch's method, smooth one long periodogram **across frequency** with a non-negative kernel. The PSD stays non-negative, and the total power is preserved.

Problem 6: Who is the problem important to and why? What would the impact be if you were successful in developing an accurate algorithm?

Who the problem matters to

- **Transmission utilities and system operators.** Galloping can cause phase-to-phase flashovers, damaged hardware, and, in severe cases, structural failure. Today a utility usually learns of an event only after the damage is done. A live alert would tell operators which span is galloping while it is happening, so they can act on it (send crews, plan around repeated faults on that line) instead of reconstructing the event afterward.
- **Line designers and maintenance engineers.** Galloping is rare and hard to observe, so there is little measured data on real events: which mode, how large, and at what wind speed. Measured mode and amplitude records from real spans would let engineers check clearance designs against actual motion and judge whether anti-galloping countermeasures are working.
- **Researchers and standards groups.** Camera-based gallop measurement has been shown to work (Yang et al.; CIGRE ISH 2015), and the IEEE P2828 working group is writing a guide for it. The published work, however, says little about the discrete-time design decisions (decimation, filter type, window length, smoothing, alert logic) or how they trade accuracy against latency.

Impact of an accurate algorithm

- **Inexpensive, non-contact monitoring.** A fixed camera becomes a gallop sensor, with no hardware installed on energized conductors.
- **Trustworthy alerts.** An alert that fires within a known latency and does **not** fire on blowout or ice shedding is one an operator can act on. False alarms matter as much as accuracy: an alert that cries wolf gets ignored.
- **Calibrated event records.** Mode number, frequency, and peak-to-peak amplitude in metres for every event, which is exactly the field data designers lack.
- **Design guidance.** Because every design choice is scored against simulator ground truth, the project shows which choices actually matter. That is useful to anyone building a similar monitor, including groups working on the P2828 guide.

Problem 8: What instrumentation or data do you have available to support this project?

Data

- **Physics-based synthetic video generator.** I built this before the proposal as Step 1 of the gallop-monitoring effort. It renders a 250 m span of Drake ACSR with 6 mm of glaze ice, plus a steel shield wire. A calibrated pinhole camera views the span broadside at 1280 × 720 and 30 fps. Gallop follows coupled Den Hartog dynamics under a scripted wind schedule ($4 \rightarrow 16$ m/s and back). Blowout, turbulent gusts, ice shedding, weather, camera shake, and sensor noise can each be switched on or off.
- **Exact per-frame ground truth.** Each run writes a CSV of modal coordinates, displacements, pixel positions at 25, 50, and 75 % of the span, amplitude envelopes, and alert labels. It also writes `metadata.json` with the camera matrices (K, R, t), conductor properties, sag, and modal frequencies. Because the modal coordinates are logged, I can compare the high-pass output with the true gallop component sample by sample. I can also check the period-matched smoother against the true f_1.
- **Scenario matrix S1–S8.** These isolate each confounder:
  - S5 (blowout only) tests the blowout residual of the high-pass designs.
  - S6 (ice shedding) tests whether a transient can cause a false alert.
  - S3 (stepped amplitude) is a direct test of the envelope smoother's step response and overshoot.
  - S8 repeats the nominal case over 10 random seeds, giving the variance of every metric.
- **Test signals.** I can inject sinusoids of known frequency and amplitude, plus steps, to verify filter responses and spectral scaling, the same way I checked the periodogram scaling in this homework.
- **Reference data.** The EPRI Orange Book provides field statistics (mode counts, orbit shape) and empirical amplitude relations (Hunt–Richards, Lilien–Havard) as sanity checks on simulated amplitudes.

Instrumentation and tools

- Python 3.10+ with NumPy, SciPy, and OpenCV on a laptop or desktop. Each filter and spectral step is written out directly and checked against `scipy.signal`.
- A function generator and oscilloscope on my home bench. These could drive and verify a small vibrating-wire rig if I pursue the bench-rig stretch goal.

What is not available

No real gallop footage with ground truth exists to my knowledge, so the quantitative evaluation relies on the simulator. I will frame results as relative comparisons between designs rather than absolute field performance.

Problem 9: How do you plan to display your results. What tables or figures will you include?

All results will be in physical units (metres, hertz, seconds) and scored against the simulator ground truth.

Figures

1. **Blowout-filter comparison:**
   - magnitude response (dB) of the Kaiser FIR, the Butterworth IIR, and the period-matched $(1-H)^2$ filter, overlaid on the specification mask, with f_1, f_2, and f_3 marked;
   - group delay of each design;
   - impulse responses;
   - a pole–zero plot for the IIR design.
2. **Time-domain example at one span station:**
   - raw displacement and the smoothed blowout estimate;
   - high-pass output overlaid on the true gallop component;
   - edge regions shaded, as in this homework.
3. **Spectrogram:** the STFT with the true modal frequencies overlaid, showing gallop growing and dying away with the wind schedule.
4. **PSD comparison over a steady-gallop interval:** raw periodogram, frequency-smoothed (Daniell) periodogram, and Welch estimate on the same axes.
5. **Envelope and alerts:**
   - estimated semi-amplitude from the boxcar-smoothed power, the Hilbert envelope, and the STFT, against the ground-truth envelope;
   - the 0.75 m and 0.50 m thresholds;
   - estimated and true alert raise/clear times.

   I will show this for the stepped-amplitude scenario (S3) and the ice-shedding scenario (S6).
6. **Latency–accuracy trade-off:** median alert latency against RMS amplitude error for each filter and smoother design and each window length.
7. **Sensitivity of the period-matched filter:** gain at f_1 against the error in the predicted f_1.

Tables

1. **Filter-specification compliance** for each design:
   - attenuation at 0.05 Hz;
   - passband ripple;
   - gain at f_1–f_3;
   - delay;
   - arithmetic per sample.
2. **Performance metrics** P1–P7 for each scenario (S1–S7) and each blowout filter, with pass/fail against the targets.
3. **Monte Carlo results (S8):** mean $\pm$ standard deviation of every metric over 10 seeds.

Problem 10: What, if anything, did you learn from this homework?

- **Normalized convolution for edge handling** At the ends of a signal, the filter is applied only to the samples that exist, and the result is divided by the sum of the taps that were used. Unlike zero padding, this keeps the DC gain at exactly 1 at the edges. I used it for every filtered output in Problems 2 and 3, and recommended it in Problem 4. It only works because $h[n] \geq 0$.

- **Periodograms** A PSD estimate made from a single FFT of the entire record, scaled by $1/(2\pi L)$ so that the area under the PSD equals the signal's average power. I used it for all

the input and output PSDs in Problems 2 and 3, in place of Welch's method, which fails on the step because its short segments are mostly all 0s or all 1s.

- **Spectral inversion** Turning a lowpass filter into a highpass filter by subtracting its output from the original signal: $h_{\text{hp}}[n] = \delta[n] - h_{\text{lp}}[n]$, so $H_{\text{hp}} = 1 - H_{\text{lp}}$. This is how I built the highpass filter in Problem 3. The triangular prototype worked best because its response never goes negative, so the highpass never overshoots unity gain.
- **Monte Carlo testing** Repeating a random experiment many times and averaging, so that a systematic error (bias) can be separated from random noise. In Problem 4, I filtered 4000 independent noise records to measure each edge method's bias and RMSE at each distance from the edge, and the results matched the theoretical predictions.
- **Noise-free ramp testing for method handling of trends at a boundary** A symmetric filter with unit DC gain passes a straight line through unchanged in the interior, so any output error on a clean ramp comes from the edge method alone. In Problem 4, this test isolated how each method handles a trend: odd reflection was exact, and periodic extension and zero padding failed badly.
- **Spectrograms of the STFT** A plot of how a signal's frequency content changes over time, made from FFTs of short, overlapping windows. It isn't used in the homework itself. I plan to use it in my gallop project (Problems 7 and 9) to track the gallop mode frequencies as they grow and die away.

Problem 11: What new questions, if any, do you have?

I have no questions at this time.